Lab Manual

Subject: Full Stack Java Programming

Suggested list of Experiments		
Sr No	**Title of Experiments**	**Technology**
01	Programs on classes and objects with Looping and Branching	**Java**
02	Programs on method and constructor overloading.	
03	Programs on various types of inheritance and Exception handling	
04	Program on Implementing Generic and HTTP servlet.	**JSP and Servlet**
05	Design a login webpage in JSP that makes validation through Database using JDBC and call the servlet for various operations	
06	Program on Implicit and Explicit objects in JSP	
07	Program to create a static website using HTML CSS and JavaScript	**HTML & CSS**
08	Program using Java Script to validate the email address entered by the user (check the presence of "@" & "." character. If this character is missing, the script should display an alert box reporting the error and ask the user to re-enter it again).	**Java Script (Any three)**
09	Program based on Document Object Model to change the background color of the web page automatically after every 5 seconds.	
10	Program to Build a Stop Watch using Java Script	
11	Program to Implement Java Script Animation	
12	Program for making use of React Hooks that displays four buttons namely, "Red", "Blue", "Green", "Yellow". On clicking any of these buttons, the code displays the message that you have selected that particular color.	**React**
13	Creating a Single Page website using the concepts in React like Hooks, Router, Props and States.	
14	Program to create a Monolithic Application using SpringBoot	**SpringBoot**
15	Program for Building RESTful APIs with spring boot	

Experiment No 1

Title:- Program on classes and objects.

Lab Outcome:-(LO1)Explore the core principles of Object-Oriented Programming (OOP) using Java such as classes, objects, Polymorphism, encapsulati**on.**

Theory:-

What is a Class?

A class in Java is a blueprint or template from which objects are created.

It defines data members (variables) and methods (functions) that represent the behavior of the object.

Example:-

```
class Student {
   int rollNo;
   String name;
   void display() {
      System.out.println("Roll No: " + rollNo + ", Name: " + name);
                  }
}
```

What is an Object?

An object is an instance of a class.

It occupies memory and has its own identity, state, and behavior.

Example:-

```
Student s1 = new Student();  // s1 is an object of Student class
```

What is a Constructor?

A constructor is a special method in Java that is used to initialize objects of a class.

It is automatically called when an object is created using the new keyword.

A constructor has the same name as the class and does not have a return type, not even void

What is the new keyword in Java?

The new keyword in Java is used to **create objects** dynamically during runtime. When you use new, it:

Allocates memory in the heap for the object.**Calls the constructor** to initialize the object.

Steps in Object-Oriented Programming:-

1. Define the class with variables and methods.
2. Create objects using the new keyword.
3. Access members using the object (objectName.variable or objectName.method()).
4. Initialize data using constructors or setter methods.
5. Retrieve or display data using getter methods or print statements.

Prerequisite:-

1. You need to download the latest version of JDK(Java Development Kit) from oracle website, and then install it on your computer
   https://download.oracle.com/java/24/latest/jdk-24_windows-x64_bin.exe

2. Yo need to download and install Notepad++ for typing,editing java codes.
   https://github.com/notepad-plus-plus/notepad-plus-plus/releases/download/v8.8.3/npp.8.8.3.Installer.x64.exe

3. After installing the above two applications ,create your own folder where you are going to save java programs, for example in C drive I will create folder java programs.
   Press the window button then type cmd and press Enter Button type cd\ then Enter button then type C: then press Enter Button then type cd java programs then Enter.

4. Validate installation of jdk by typing javac
5. Validate installation of jdk by typing javac
6. Now your first program Class1.java you can type in notepad++ and you save it in C:\javaprograms

```
C:\Users\admin>cd\

C:\>cd java programs

C:\java programs>javac
Usage: javac <options> <source files>
where possible options include:
  @<filename>                  Read options and filenames from file
  -Akey[=value]                Options to pass to annotation processors
  --add-modules <module>(,<module>)*
        Root modules to resolve in addition to the initial modules,
        or all modules on the module path if <module> is ALL-MODULE-PATH.
  --boot-class-path <path>, -bootclasspath <path>
        Override location of bootstrap class files
  --class-path <path>, -classpath <path>, -cp <path>
        Specify where to find user class files and annotation processors
  -d <directory>               Specify where to place generated class files
  -deprecation
        Output source locations where deprecated APIs are used
  --enable-preview
        Enable preview language features.
        To be used in conjunction with either -source or --release.
  -encoding <encoding>         Specify character encoding used by source files
  -endorseddirs <dirs>         Override location of endorsed standards path
  -extdirs <dirs>              Override location of installed extensions
```

Source Code Typed in Notepad++ :-

```
public class Class1
{
  int n;
  String s;
  public Class1()
  {
      System.out.println("Default Constructor")
      System.out.println("n="+n+"s="+s);

  }
  public void setN(int n1)
  {
   n=n1;
  }
  public void setS(String s1)
  {
     s=s1;
  }
  public int getN()
  {
     return n;
  }
  public String getS()
  {
    return s;
  }
  public static void main(String arg[])
  {
       Class1 c1=new Class1();
       c1.setN(12);
       c1.setS("Hello");
       System.out.println("n="+c1.getN());
       System.out.println("s="+c1.getS());
  }
}
```

Source Code in Notepad

```
public class Class1
{
 int n;
 String s;
 public Class1()
 {
        System.out.println("Default Constructor") ;
        System.out.println("n="+n+"s="+s);
 }
 public void setN(int n1)
 {
  n=n1;
 }
 public void setS(String s1)
 {
   s=s1;
 }
 public int getN()
 {
   return n;
 }
 public String getS()
 {
  return s;
 }
```

```
public static void main(String arg[])
{
        Class1 c1=new Class1();
        c1.setN(12);
        c1.setS("Hello");
        System.out.println("n="+c1.getN());
        System.out.println("s="+c1.getS());
}
}
```

Procedure to compile and execute java code:-

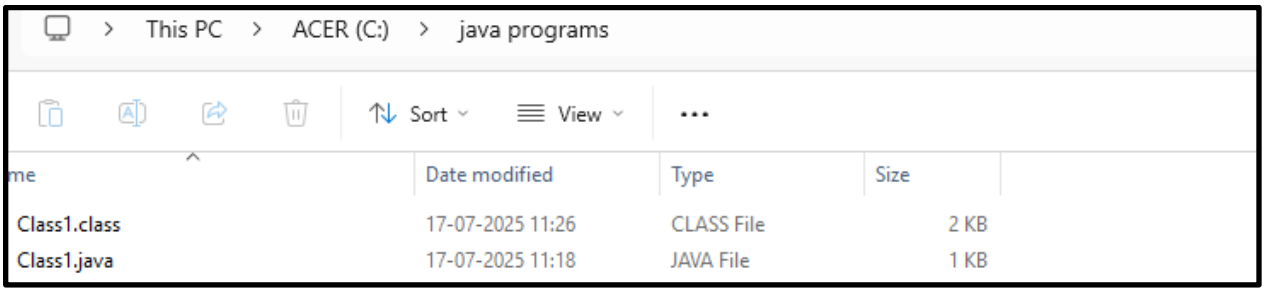
Compilation of Code:-

How to compile above java program using javac command

Javac Class1.java

```
C:\java programs>javac Class1.java
```

If compiled successfully then Class1.class get generated in the same directory.



Execution of Code:-

Java Class1

```
C:\java programs>java Class1
```

java command expects Just the name of the class containing the public static void main(String[] args) method.

Output:-

```
Default Constructor
n=0s=null
n=12
s=Hello
```

Explanation

```
class Class1 {

   int n;       // instance variable: integer

   String s;    // instance variable: string
```

These two variables belong to each object of the class. They are **not initialized** explicitly, so they get **default values**:

int n =0
String s = null

```
public Class1() {

  System.out.println("Default Constructor");

  System.out.println("n=" + n + " s=" + s);

}
```

This is a **default constructor** (no parameters).

It runs automatically when an object of Class1 is created.

It prints default values of n and s, which will be:

```
public void setN(int n1) {

   n = n1;

}

public void setS(String s1) {
```

```
    s = s1;
}
```

These are **setter methods**: used to assign values to the instance variables n and s

```
public int getN() {
    return n;
}
public String getS() {
    return s;
}
```

These are **getter methods**: used to access the values of n and s

```
public static void main(String arg[]) {
    Class1 c1 = new Class1();      // Constructor is called
    c1.setN(12);                   // set n = 12
    c1.setS("Hello");              // set s = "Hello"
     System.out.println("n=" + c1.getN());  // Output: n=12
    System.out.println("s=" + c1.getS());  // Output: s=Hello
}
```

Final Output

```
Default Constructor
n=0 s=null
n=12s=Hello
```

Key Concepts Learned:-

Constructor	Special method that runs when an object is created.
Setter methods	Used to assign values to private or instance variables.

Getter methods Used to retrieve values from private or instance variables.
Object Instance of the class used to access methods and variables.

Why must the file be named Class1.java?

In Java, the filename must match the public class name exactly, including capitalization.

If you save the program as Test.java, you'll get this error:

class Class1 is public, should be declared in a file named Class1.java

If a class is not public, it can be saved with any name, but then you need to specify the name of class which contains the main function .

Viva Question:-

What is a class in Java?

What is an object?

What is the difference between a constructor and a method?

What is the use of the new keyword?

Can we have multiple classes in a single .java file?

What happens if the filename doesn't match the public class name?

What is the role of the main() method in Java?

What is the difference between Procedure Oriented Programming and Object Oriented Programming?

Practical Exercise:-

Create a Class Student with the following things.
With private data members: rollNo, name, and marks.
Write setter and getter methods for each field.
Create an object and set the values using setters and display them using getters.

Experiment No 2

Title:-Program on method and constructor overloading.

Aim:-To write a Java program that demonstrates: Constructor Overloading – using different constructors in the same class. Method Overloading – using methods with the same name but different parameter lists.

Theory:-

Overloading is a feature of compile-time polymorphism in Java, where multiple methods or constructors can have the same name but different parameter lists.

There are two types of overloading in Java:

1. Constructor Overloading
2. Method Overloading

- **Constructor overloading** means having multiple constructors in the same class, each with a different number or type of parameters.

Rules:

- Constructors must have the same name as the class.
- No return type (not even void).
- Must differ by number or types of parameters.

- **Method overloading** means defining multiple methods with the same name in a class, but with different parameter lists.

Rules:

- All overloaded methods must have the **same name**.
- Method overloading is based on **parameter count, type, or order** — at least one of these must differ.
- All overloaded methods must be defined in the **same class** or **inherited** from a superclass.
- Overloaded methods can have **different access modifiers** (public, private, etc.), but it's not a rule — only allowed.

Command Line Arguments:-

Command line arguments are the values passed to a Java program during execution from the command prompt (terminal). They allow the user to provide input at runtime.

In main function args is a String array,Each element of args[] contains one command-line argument as a String. public static void main(String[] args)

Example :-

java Class2 10 20

Here Argument 0: 10

Argument 1: 20

Java Source Code for Method and Constructor Overloading :-

```
class Student
{
 String name;
 int roll_no;
 double marks;
 public Student()
 {
  System.out.println("Default Constructor") ;
  System.out.println("Default value of name and roll_no"+name+roll_no);
 }
 public Student(String name,int roll_no)
 {
```

```
  this.name=name;
  this.roll_no=roll_no;
 }
 public Student(String name,int roll_no,double marks)
 {
  this.name=name;
  this.roll_no=roll_no;
  this.marks=marks;
 }
}
class B
{
 public void area(int r)
 {
 System.out.println("Area of circle"+(3.14*r*r));
 }
 public void area(int l,int b)
 {
            System.out.println("Area of rectangle"+(l*b));
 }
}
```

```
public class Class2

{

 public static void main(String arg[])

 {

        Student s1,s2,s3;   //declaration of reference variable

  s1=new Student();

        s2=new Student("akshay",1);

        s3=new Student("nilesh",2,90.2);

        B b1;

        b1=new B();

        b1.area(10);

        b1.area(10,20);

        //If you want to pass parameters from user

        int r;

        int l,b;

        r=Integer.parseInt(arg[0]);

        l=Integer.parseInt(arg[1]);

        b=Integer.parseInt(arg[2]);

        b1.area(r);

        b1.area(l,b);

 }}
```

Output:-

```
C:\java programs>java Class2  9 3 4
Default Constructor
Default value of name and roll_nonull0
Area of circle314.0
Area of rectangle200
Area of circle254.34
Area of rectangle9
```

Explanation:-

In this class:
Student() → Default constructor, no parameters.
Student(String, int) → Initializes name and roll_no.
Student(String, int, double) → Initializes name, roll_no, and marks.
Java chooses the appropriate constructor based on the **number and type of arguments** passed during object creation.

Method Overloading happens when multiple methods in the same class share the same name but have different parameter lists.

In this class:

area(int r) → Calculates area of a circle

area(int l, int b) → Calculates area of a rectangle

Java selects the correct area() method at compile time based on how many and what kind of arguments are passed.

Objects s1, s2, and s3 demonstrate constructor overloading:

Each one calls a different constructor depending on how many arguments are passed.

Object b1 calls the area() method in two overloaded forms:

Once with 1 argument (circle)

Once with 2 arguments (rectangle)

It also shows how to use command-line arguments to pass values from the terminal.

Viva Questions:-

What is method overloading?

What is constructor overloading?

Exercise:-

Create a class Employee that demonstrates constructor overloading:

Data members: empId, empName, salary

Constructors:

No-argument constructor (initialize with default values)

Two-argument constructor (empId, empName)

Three-argument constructor (empId, empName, salary)

A method display() to print employee details

Create another class MathOperations with overloaded methods:

multiply(int a, int b) → product of two integers

multiply(double a, double b) → product of two decimals

multiply(int a, int b, int c) → product of three integers

In the main() method:

Create multiple Employee objects using different constructors

Call different multiply() methods with different parameters

Experiment No. 3

Title:-Programs on various types of inheritance and Exception handling

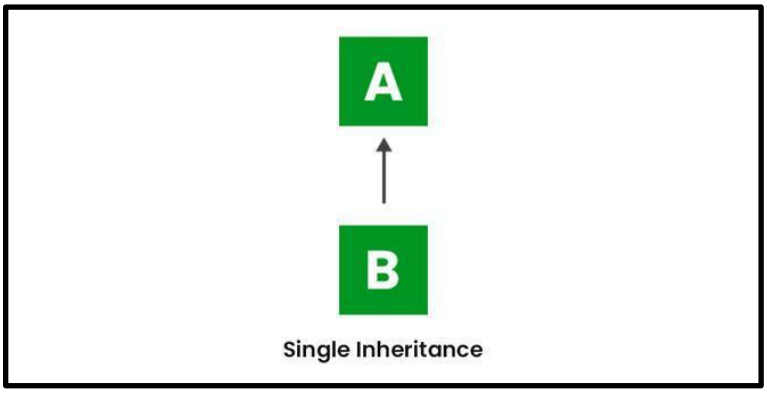
Aim:- write a Java program to demonstrate the concepts of: Single Inheritance, Multilevel Inheritance, Method Overriding (Runtime Polymorphism) and Multiple Inheritance using Interfaces

Theory:-

Inheritance:-

Inheritance is a fundamental concept in OOP(Object-Oriented Programming). It is the mechanism in Java by which one class is allowed to inherit the features(fields and methods) of another clas

Single Inheritance

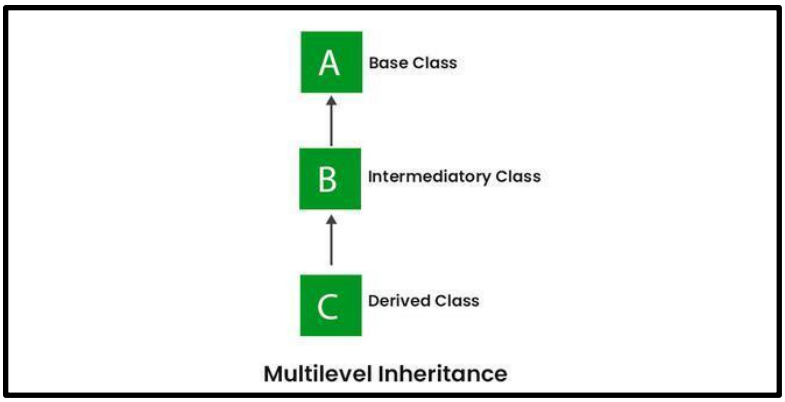


In single inheritance, a subclass inherits the properties and methods of a single superclass.

Syntax: class B extends A

Enables code reuse and a hierarchical classification.

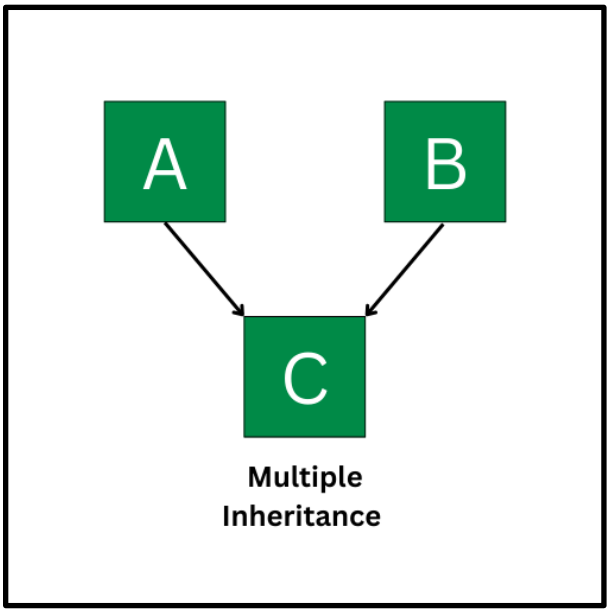
Multilevel Inheritance



Multilevel inheritance occurs when a class is derived from a class which is already derived from another class.

class C extends B where B is already superclass of A B extends A

Multiple Inheritance (via Interfaces)



Java does not support multiple inheritance with classes but allows it using **interfaces** to avoid ambiguity. Java does not support multiple class inheritance but allows multiple interface implementation using the implements keyword.

Interfaces D1 and D2

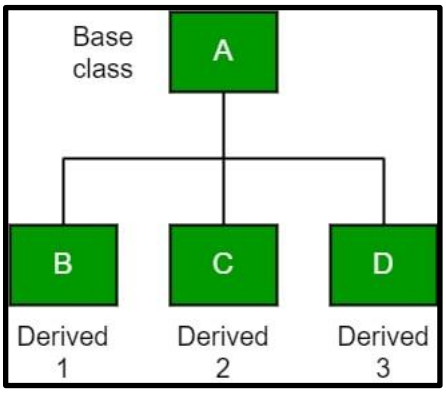
class D extends A implements D1, D2

Constructors and Constructor Chaining

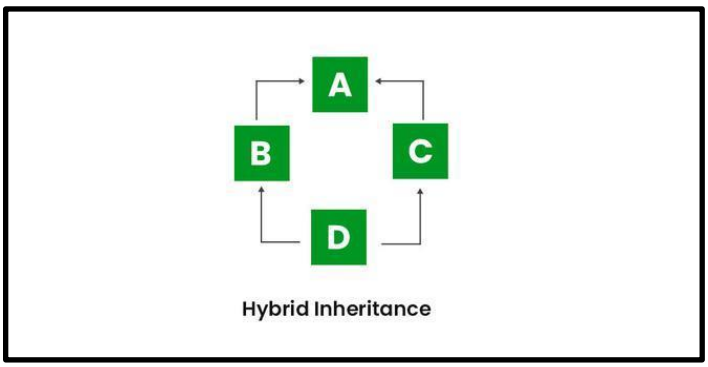
A constructor is a special method that is automatically called when an object is created. In Java, constructors of superclasses are automatically called when a subclass object is instantiated.Polymorphism

Hierarchical Inheritance

In Hierarchical Inheritance, one class serves as a superclass (base class) for more than one subclass. In the below image, class A serves as a base class for the derived classes B, C, and D.



Hybrid Inheritance



It is a mix of two or more of the above types of inheritance. Since Java doesn't support multiple inheritance with classes, hybrid inheritance involving multiple inheritance is also not possible with classes.

Method Overriding

Overriding in Java occurs when a subclass or child class implements a method that is already defined in the superclass or base class. When a subclass provides its own version of a method that is already defined in its superclass, we call it method overriding. The subclass method must match the parent class method's name, parameters, and return type.

Rules for Overriding:

- Name, parameters, and return type must match the parent method.
- Java picks which method to run, based on the actual object type, not just the variable type.
- Static methods cannot be overridden.

Occurs when a subclass provides a specific implementation of a method already defined in its superclass.

- Same method signature
- Achieves **runtime polymorphism**

Java Code:-

```
class A {

  private int i;

  private String s;

  public A() {

        System.out.println("Inside Default Constructor of A");

  }

  public void setI(int i) {

        this.i = i;

  }

  public void setS(String s) {

        this.s = s;
```

```
  }
  public int getI() {
        return i;
  }
  public String getS() {
        return s;
  }
  public void display() {
        System.out.println("display function Inside class A");
  }
}
class B extends A {
  public B() {
        System.out.println("Inside Default Constructor of B");
  }
  public void display() {
        System.out.println("display function Inside class B");
  }
}
class C extends B {
  public C() {
```

```
        System.out.println("Inside constructor of C");
    }
    public void display() {
        System.out.println("display function Inside class C");
    }
}
interface D1 {
    public void display1();
}
interface D2 {
    public void display2();
}
class D extends A implements D1, D2 {
    public void display1() {
        System.out.println("display1 of interface D1");
    }
    public void display2() {
        System.out.println("display2 of interface D2");
    }
}
public class Class3 {
```

```
  public static void main(String arg[]) {

        System.out.println("Calling constructor of A");

        A a = new A();

        System.out.println("Calling constructor of B");

        B b = new B();



        System.out.println("Calling constructor of C");

        C c = new C();

        System.out.println("Calling constructor of D");

        D d = new D();

        d.display();

        d.display1();

        d.display2();

        a.setS("akshay");

        a.setI(12);

        a.display();

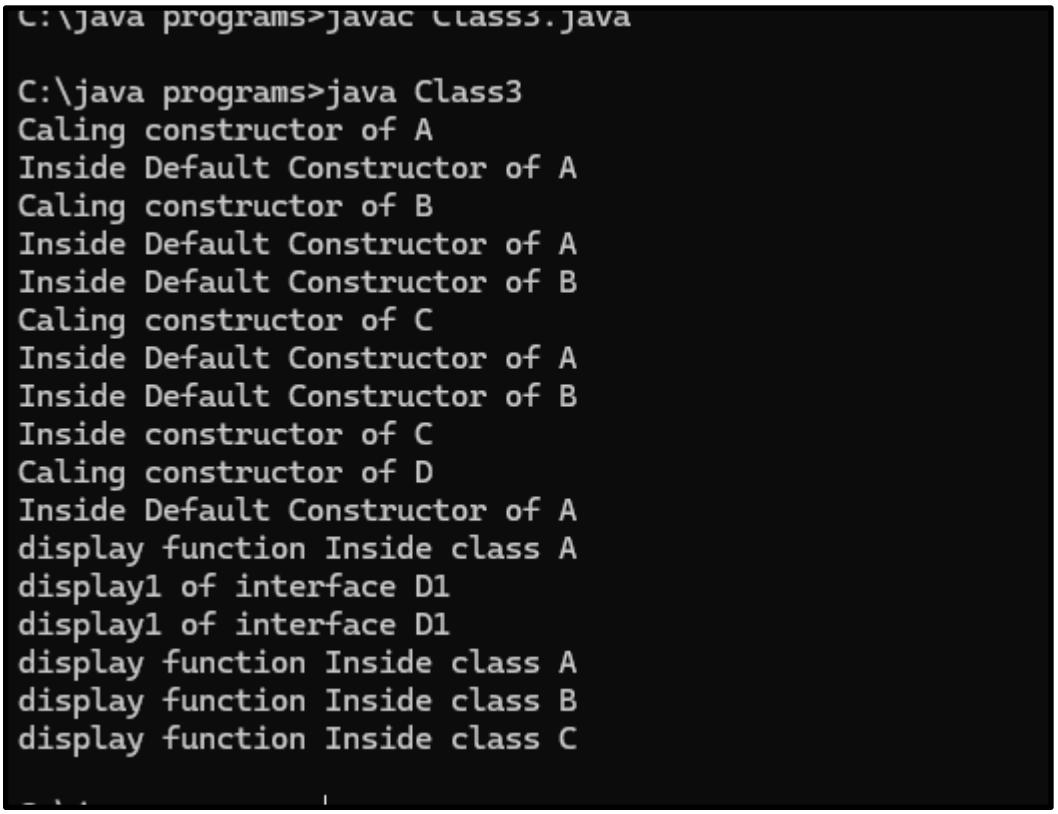
        b.display();

        c.display();

  }

}
```

Output:-



Explanation of Key Concepts

Class A (Base Class)

```
class A {
   private int i;
   private String s;
   ...
   public void display() {
            System.out.println("display function Inside class A");
   }
}
```

i and s are private → accessible only via get/set methods.

display() is defined here and gets overridden in child classes.

Class B extends A (Single Inheritance + Overriding)

```
class B extends A {
  public void display() {
        System.out.println("display function Inside class B");
  }
}

interface D1 {
  public void display1();
}

interface D2 {
  public void display2();
}
```

//Defines the methods display1() and display2()

```
class D extends A implements D1, D2 {
  public void display1() {
        System.out.println("display1 of interface D1");
  }

  public void display2() {
        System.out.println("display2 of interface D2");
```

```
  }
}
```

Inherits from A,Implements both interfaces.Defines the methods display1() and display2()

VIVA Questions:-

What is inheritance? What are the types of Inheritance in java?

What is Interface in java ? What is the use of Interface in Multiple Inheritance of java?

What is method overriding in java?

Practical Exercise:-

Write a java program which consists of Animal class with function public void eat() and then there should be a child class Dog which will override eat function of animal.

Experiment No. 4

Title:- Program on implementing Generic and HTTP servlet.

Description:

Generic and HTTP Servlets are both Java-based server-side components, but they differ in their scope and functionality. GenericServlet provides a basic, protocol-independent framework, while HttpServlet is specifically designed for handling HTTP requests and responses, making it more suitable for web applications.

GenericServlet:

- Implements the core Servlet interface and provides a basic framework for handling requests.
- Is protocol-independent, meaning it can potentially handle any type of request.
- Requires overriding the service() method to process requests.
- Aims to simplify servlet development by providing a simpler API for handling basic request processing.
- Is an abstract class, meaning it cannot be directly instantiated.
- Provides implementations for lifecycle methods like init() and destroy().

HttpServlet:

- Extends GenericServlet and is specifically designed for handling HTTP requests.
- Provides built-in support for HTTP methods like GET, POST, PUT, DELETE, etc.
- Overrides methods like doGet(), doPost() to handle specific HTTP request types.
- Offers more convenience and features for web development, such as handling HTTP headers and working with cookies.
- Is the most commonly used servlet type for web applications.
- Is also an abstract class.

Protocol Dependence:

GenericServlet is protocol-independent, while HttpServlet is designed for the HTTP protocol.

Functionality:

HttpServlet provides specific features for handling HTTP requests and responses, while GenericServlet offers a more general framework.

Usage:

HttpServlet is more commonly used in web applications, while GenericServlet can be used for other types of request-response based systems.

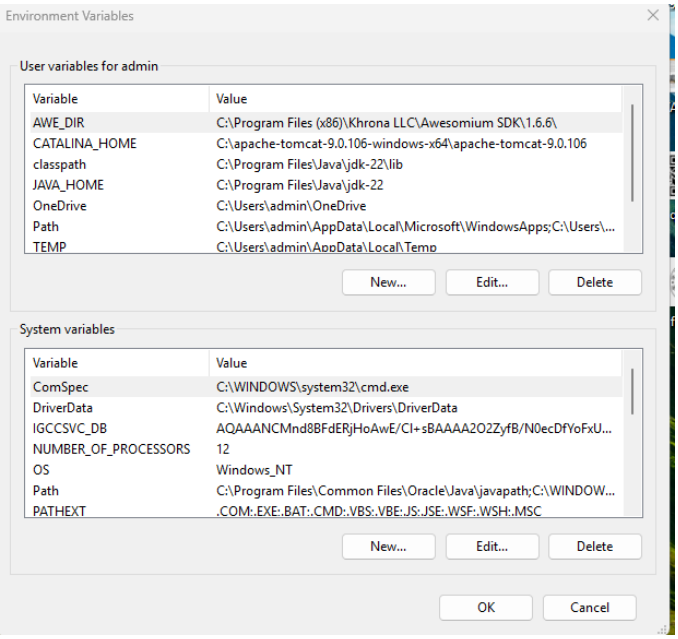
Implementation:

To compile and run servlet, we require Apache Tomcat Server.

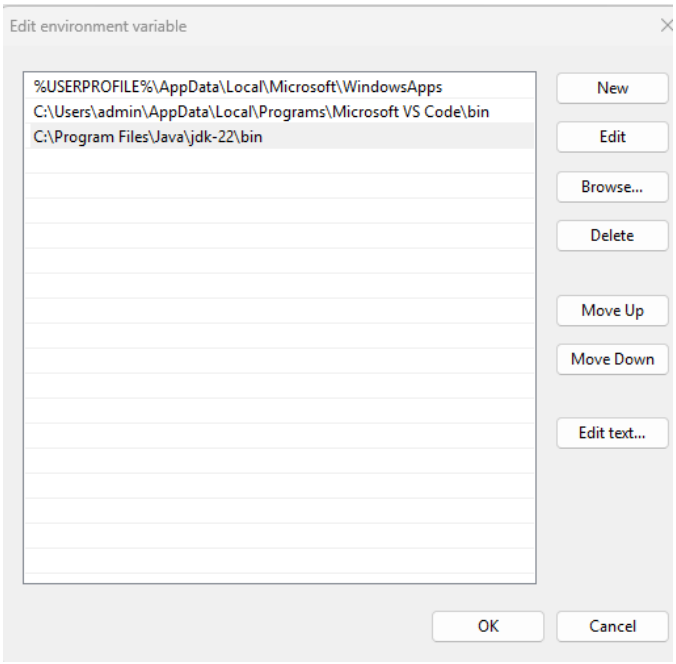
Step 1: Download apache-tomcat-9.0.106-windows-x64.zip file from https://tomcat.apache.org/download-90.cgi website

Step 2: Unzip file in C drive

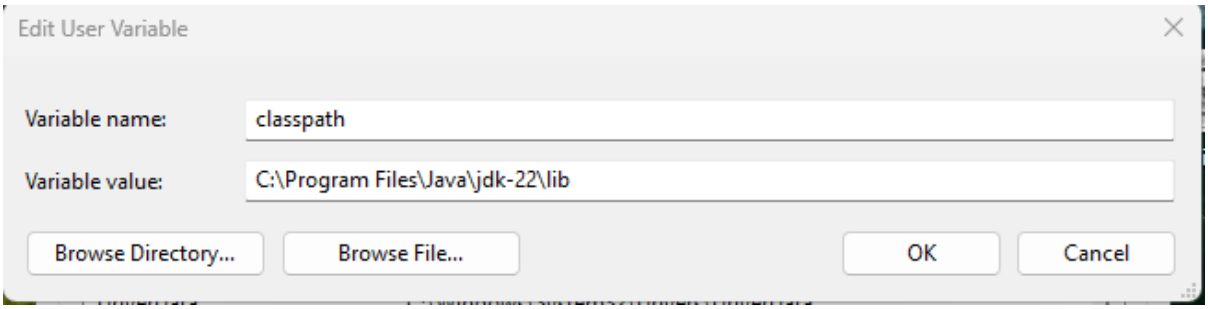
Step 3:Open Environment variable:



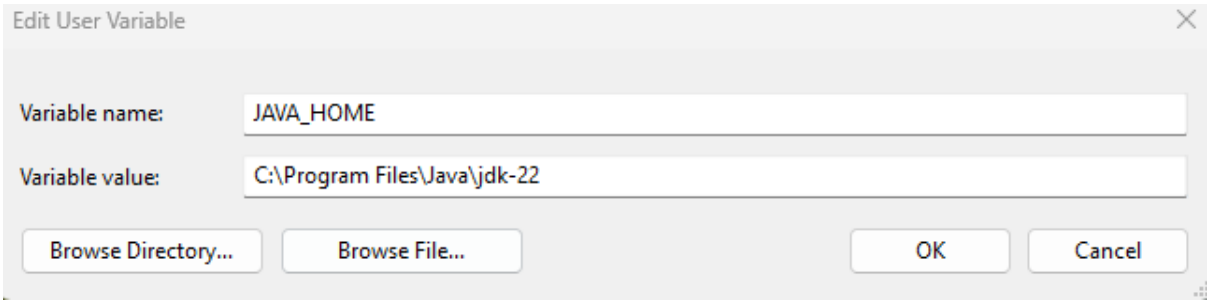
Step 4: Select “Path” User variable and click on “Edit” button. Click “New” and Add jdk bin folder path and click “OK”.



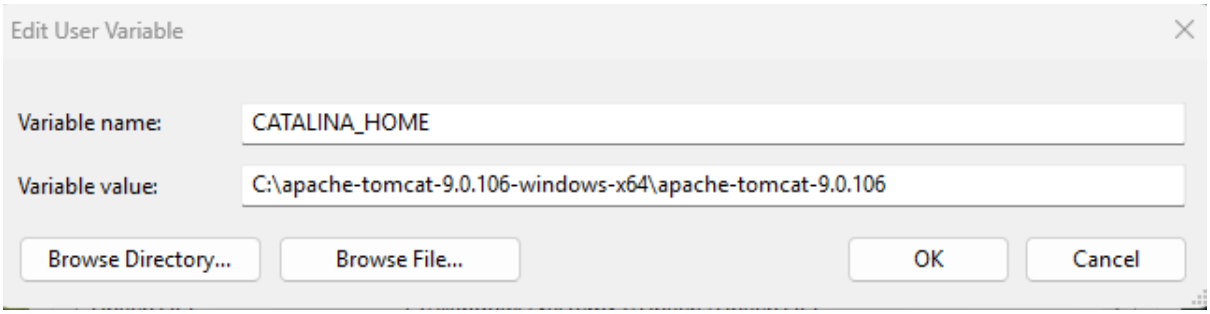
Step 5: Click “New” button of User variable and add jdk lib classpath as follows:



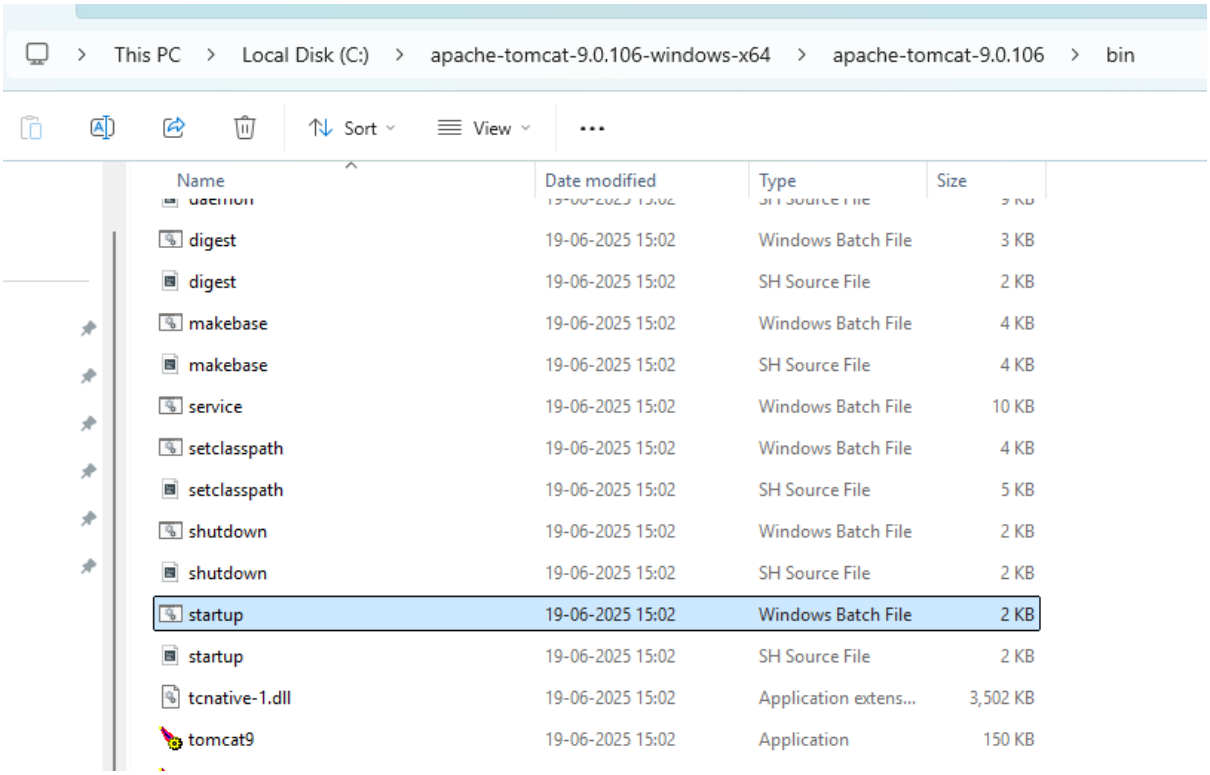
Step 6: Click “New” button of User variable and add jdk path as follows



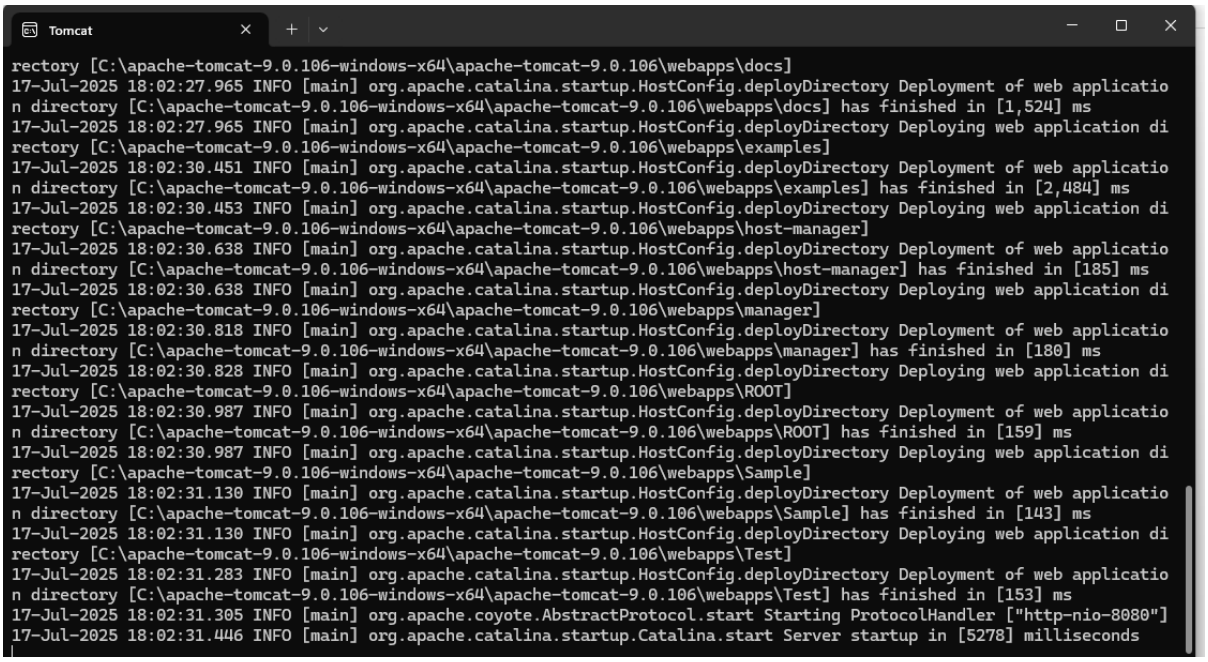
Step 7: Click “New” button of User variable and add Tomcat server path as follows



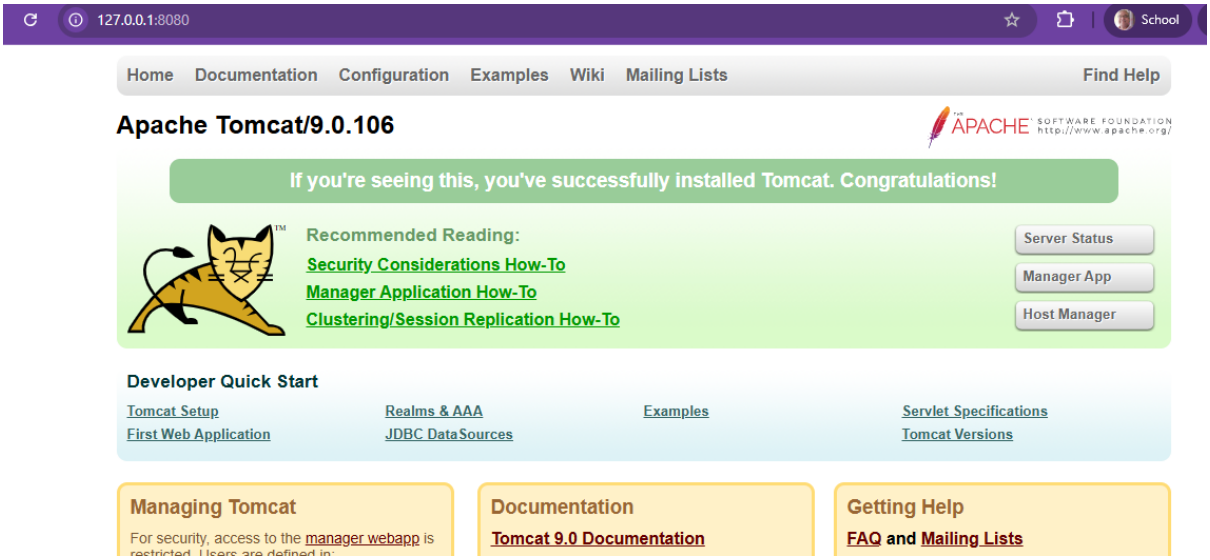
Step 8: Open Windows explore and go to C:\apache-tomcat-9.0.106-windows-x64\apache-tomcat-9.0.106\bin folder and double click on startup.bat file as shown:



Step 9: Apache Tomcat server will start as shown below: Last line will be “Server startup in ___ milliseconds.



Step 10: Open browser and type url as http://127.0.0.1:8080/ and enter, the home page of Tomcat server will open as follows: Here 127.0.0.1 is localhost and 8080 is the port number.



Step 11: Now, first the webapp is to be created. For that, go to C:\apache-tomcat-9.0.106-windows-x64\apache-tomcat-9.0.106\webapps folder and create folder as “Sample”. Inside that folder create “WEB-INF” folder and inside that create web.xml file.

For creating web,xml file, copy this file from C:\apache-tomcat-9.0.106-windows-x64\apache-tomcat-9.0.106\webapps\manager\WEB-INF and paste it in your “WEB-INF” folder.

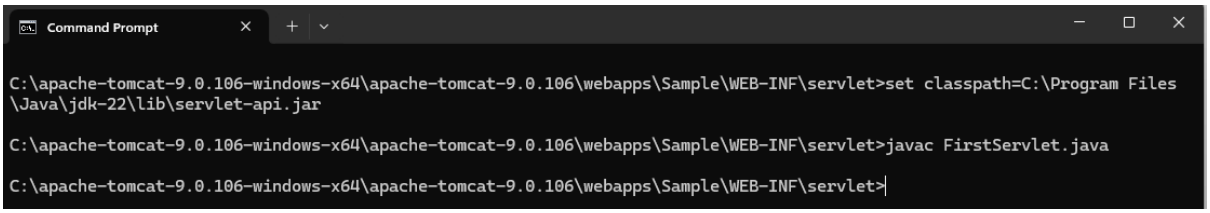
Step 12: Create folder “servlet” in Sample\WEB-INF folder and Write FirstServlet.java program as follows:

```
import java.io.*;
import javax.servlet.*;
public class FirstServlet extends GenericServlet

{
   public void service(ServletRequest req,
                       ServletResponse resp)
      throws ServletException, IOException
   {
      resp.setContentType("text/html");
      PrintWriter pw = resp.getWriter();
      pw.println("<html>");
      pw.println("<head><title>My first Servlet</title></head>");
      pw.println("<body>");
      pw.println("<h2>Welcome To Servlet World!</h2>");
      pw.println("</body>");
      pw.println("</html>");
      pw.close();
   }
}
```

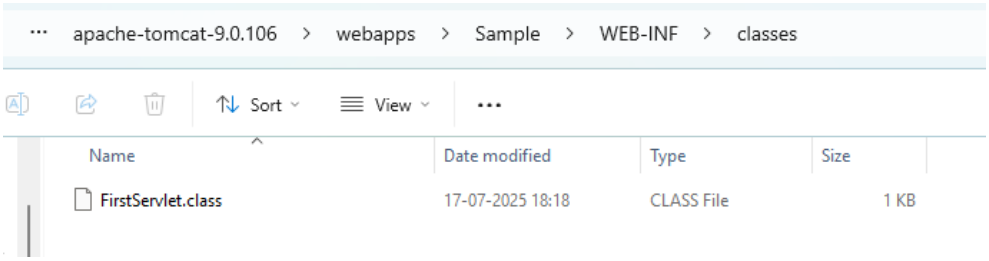
Step 13: To compile the program and get class file, copy servlet-api.jar file from C:\apache-tomcat-9.0.106-windows-x64\apache-tomcat-9.0.106\lib and store it in C:\Program Files\Java\jdk-22\lib folder.

Open command prompt, give classpath and compile servlet as shown below:



FirstServlet.class will get created.

Step 14: Create "classes" folder and keep this FirstServlet.class file inside it as shown below:



Step 15: Edit "web.xml" file as shown below: (Note: Other part of code, delete it.)

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
                      http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd"
  version="4.0"
  metadata-complete="true">

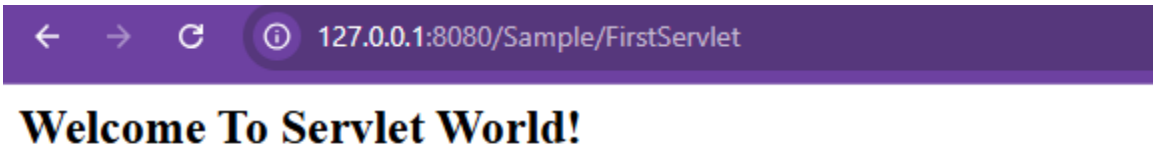
    <servlet>
        <servlet-name>FirstServlet</servlet-name>
        <servlet-class>FirstServlet</servlet-class>
    </servlet>

    <servlet-mapping>
        <servlet-name>FirstServlet</servlet-name>
        <url-pattern>/FirstServlet</url-pattern>
    </servlet-mapping>
</web-app>
```

Step 16: Open web browser and type url as:

http://127.0.0.1:8080/Sample/FirstServlet

The output of servlet file will be shown:



Welcome To Servlet World!

Experiment No 5

Title:- Program a login webpage in JSP that makes validation through Database using JDBC and call the servlet for various operations.

Prerequisites

- Apache Tomcat server
- MySQL database (users table)
- JDBC MySQL connector (JAR file in lib)

Steps for Implementation:

Step 1: Install MySQL, go to https://dev.mysql.com/downloads/installer/

And download mysql-installer-web-community-8.0.42.0

Double click and it will get install.

Step 2: To open MySQL, open command prompt and go to folder

C:\Program Files\MySQL\MySQL Server 8.0\bin

and give following command

mysql -u root -p

following screen will appear:

```
C:\Program Files\MySQL\MySQL Server 8.0\bin>mysql -u root -p
Enter password: ****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 8.0.42 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Step 2: **Create MySQL Database Table**

```
CREATE DATABASE login_db;

USE login_db;


CREATE TABLE users (
   id INT PRIMARY KEY AUTO_INCREMENT,
   username VARCHAR(50) NOT NULL,
   password VARCHAR(50) NOT NULL
);


INSERT INTO users (username, password) VALUES ('admin', 'admin123');
```

The output is as follows:

```
mysql> CREATE DATABASE login_db;
Query OK, 1 row affected (0.07 sec)

mysql> USE login_db;
Database changed
mysql> CREATE TABLE users (
    ->     id INT PRIMARY KEY AUTO_INCREMENT,
    ->     username VARCHAR(50) NOT NULL,
    ->     password VARCHAR(50) NOT NULL
    -> );
Query OK, 0 rows affected (0.06 sec)
```

```
mysql> INSERT INTO users (username, password) VALUES ('admin', 'admin123');
Query OK, 1 row affected (0.02 sec)

mysql>
```

Step 2: Write “Login.jsp” file and save file in C:\apache-tomcat-9.0.106-windows-x64\apache-tomcat-9.0.106\webapps\Sample folder as follows:

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>

<html>

<head>

   <title>Login Page</title>

</head>

<body>

   <h2>Login Form</h2>

   <form action="LoginServlet" method="post">

      Username: <input type="text" name="username" required /><br><br>

      Password: <input type="password" name="password" required /><br><br>

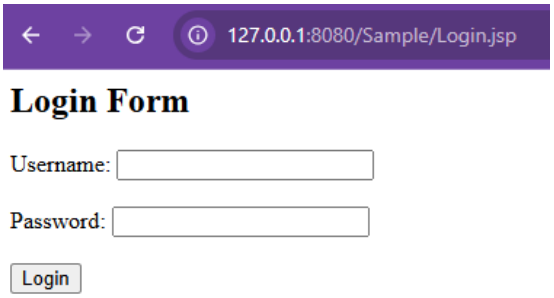
      <input type="submit" value="Login" />

   </form>

</body>

</html>
```

Step 2: Open browser and type URL as http://127.0.0.1:8080/Sample/Login.jsp



Step 3: Servlet: LoginServlet.java

Save the file in Tomcat/webapps/Sample/WEB-INF/servlet folder

import java.io.IOException;

```
import java.sql.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;

@WebServlet("/LoginServlet")
public class LoginServlet extends HttpServlet {
    private static final String DB_URL = "jdbc:mysql://localhost:3306/login_db";
    private static final String DB_USER = "root";
    private static final String DB_PASSWORD = "your_password"; // Replace with your MySQL password

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {

        String uname = request.getParameter("username");
        String pwd = request.getParameter("password");

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection conn = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
            PreparedStatement ps = conn.prepareStatement("SELECT * FROM users WHERE username=? AND password=?");
            ps.setString(1, uname);
            ps.setString(2, pwd);
```

```
            ResultSet rs = ps.executeQuery();


            if (rs.next()) {
                request.setAttribute("username", uname);
                RequestDispatcher rd = request.getRequestDispatcher("welcome.jsp");
                rd.forward(request, response);
            } else {
                RequestDispatcher rd = request.getRequestDispatcher("error.jsp");
                rd.forward(request, response);
            }


            rs.close();
            ps.close();
            conn.close();
        } catch (Exception e) {
            throw new ServletException("Login Failed", e);
        }
    }
}
```

Compile the file as follows:



And save LoginServlet.class file in Tomcat\webapps\Sample\WEB-INF\classes folder.

Step 4: Welcome Page: welcome.jsp, Save file in Tomcat\webapps\Sample folder

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>

<html>

<head><title>Welcome</title></head>

<body>

   <h2>Welcome, <%= request.getAttribute("username") %>!</h2>

   <p>You have successfully logged in.</p>

</body>

</html>
```

Step 5: Error Page: error.jsp, Save file in Tomcat\webapps\Sample folder

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>

<html>

<head><title>Login Failed</title></head>

<body>

   <h2>Login Failed</h2>

   <p>Invalid username or password. Try again.</p>

   <a href="login.jsp">Back to Login</a>

</body>

</html>
```

Step 5: Edit web.xml for Web Deployment as follows:

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee" version="3.1">

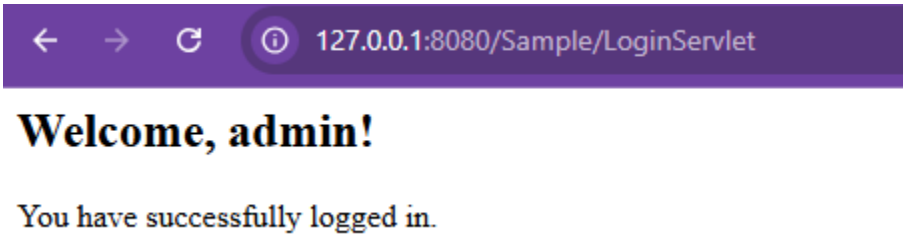
   <servlet>
```

```
        <servlet-name>LoginServlet</servlet-name>
        <servlet-class>com.example.LoginServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>LoginServlet</servlet-name>
        <url-pattern>/LoginServlet</url-pattern>
    </servlet-mapping>
</web-app>
```

Step 6: Restart the Tomcat server by clicking shutdown.bat file and again click startup.bat file

1. Visit http://localhost:8080/Sample/Login.jsp
2. Try:
    - Username: admin
    - Password: admin123

The output is shown as follows:



Experiment No. 6

Experiment 6

Title:- Program on Implicit and Explicit objects in JSP.

Description:

JSP, which stands for JavaServer Pages, is a server-side technology used to create dynamic web pages. It allows developers to embed Java code within HTML pages using special JSP tags, enabling the generation of dynamic content based on user requests and other factors.

Here's a more detailed explanation:

- Dynamic Web Pages:

  JSP is designed to build dynamic web pages, meaning the content that users see can change based on various factors like user input, database information, or time.

- Embedded Java Code:

  JSP allows developers to write Java code directly within HTML pages using JSP tags. This code is executed on the server, and the resulting output is sent to the client's browser as HTML.

- Part of Java EE:

  JSP is a core component of the Java EE (Enterprise Edition) platform, which provides a comprehensive set of technologies for building enterprise-level web applications.

- Relationship to Servlets:

  JSP can be seen as an extension of Java servlets. While servlets are Java classes, JSP provides a more convenient way to create dynamic web pages by embedding Java code within HTML-like syntax.

- JSP Compiler:

  JSP pages are translated into Java servlets by a JSP compiler. This compiled servlet is then executed by the web server to generate the dynamic content.

- Example:

  A simple JSP page might include Java code to display the current date and time, or to retrieve and display data from a database.

Components of a Java Server Page

- Directives
- Actions
- Implicit Objects
- JSP Scripting

Directives

Directive Elements	
Element	**Description**
<%@ page ... %>	Defines page-dependent attributes, such as scripting language, error page, and buffering requirements
<%@ include ... %>	Includes a file during the translation phase
<%@ taglib ... %>	Declares a tag library, containing custom actions, used in the page

<%@ page language = "java" %>

The attributes of page Directive:

Language = "scripting language"	Extends="classname"
Import="importlist"	Session="true/false"
Buffer="none/size in kb"	autoFlush="true/false"
isThreadSafe="true/false"	Info="text"
Errorpage="error_url"	isErrorPage="true/false"
contentType="ctinfo"	

<%@ include file="relativeURLSpec" %>

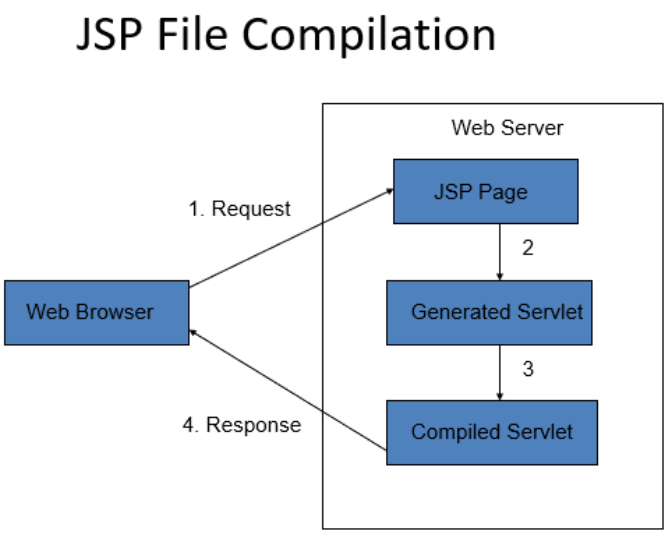
<%@ taglib uri="taglibraryURI" prefix = "tagprefix" %>

Implicit Objects

Total 9 implicit objects which are as follows

1. *request*: This is the object of **HttpServletRequest** class associated with the request.
2. *response*: This is the object of **HttpServletResponse** class associated with the response to the client.
3. *config*: This is the object of **ServletConfig** class associated with the page.
4. *application*: This is the object of **ServletContext** class associated with the application context.
5. *session*: This is the object of **HttpSession** class associated with the request.
6. *page context*: This is the object of **PageContext** class that encapsulates the use of server-specific features. This object can be used to find, get or remove an attribute.
7. *page object*: The manner we use the keyword **this** for current object, page object is used to refer to the current translated servlet class.
8. *exception*: The exception object represents all errors and exceptions which is accessed by the respective jsp. The exception implicit object is of type **java.lang.Throwable**.
9. out: This is the **PrintWriter** object where methods like print and println help for displaying the content to the client.

The JSP file will compile as follows:

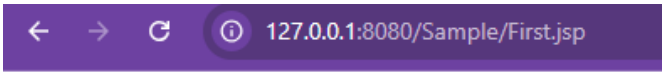


Step 1: Write first JSP program as follows:

```
<html>
<head>
   <title> First Program </title>
```

```
</head>
<body>
        <h1> Output of JSP file </h1>
        <%
           out.println("Hello World");
        %>
<body>
<html>
```

Save “First.jsp” file in C:\apache-tomcat-9.0.106-windows-x64\apache-tomcat-9.0.106\webapps\Sample folder and open browser and type url as http://127.0.0.1:8080/Sample/First.jsp



Output of JSP file

Hello World

Step 2: Go to

https://github.com/yogeshpingle/JSP_Servlet/tree/main/Sample

You will find all jsp programs, out of which you can try any 2/3 programs.

2dispDate.jsp

3greetings.jsp

4pagecount.jsp

Experiment No 7

Aim: Program to create a static website using HTML and CSS

Objectives: To create a static website using HTML,CSS and JavaScript

Prerequisites: It is required that students should know the basic HTML, Style sheet, and Java Script syntax and methods

❖ Steps to Create Static Pages

Step 1: Firstly, we will create a folder with the project name and create the HTML(e.g, index.html), CSS (e.g, style.css), JavaScript files, Images folder, and a separate file for the responsive CSS if required.

Step 2: By using various HTML tags like section, header, meta, title, head, div, I, mg, etc to structure the web page. and includes a header, navigation bar, main content area, and a script tag for our JavaScript file.

Step3: Add some Style using CSS to the HTML code to make our frontend look good. We'll choose colors and layouts that look modern and inviting. Add style mainly to the image and beautify the header.

Step4: Add the JavaScript function, like updating the content based on what the user clicks.

Step5: You can create a navigation system that dynamically loads content based on user interactions.

- **Code**
- **HTML: index.html**

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>My Static Website</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <header>
    <h1>Welcome to My Static Website</h1>
    <nav>
      <ul>
        <li><a href="#">Home</a></li>
        <li><a href="#">About</a></li>
        <li><a href="#">Contact</a></li>
      </ul>
    </nav>
  </header>
  <main>
    <section>
      <h2>Hello, Dear Student!</h2>
      <p>This is a sample static website built using HTML, CSS, and JavaScript.</p>
      <button onclick="showMessage()">Click Me</button>
      <p id="message"></p>
    </section>
  </main>
```

```
  <footer>
    <p>© 2025 FullStackJavaSyllusSetter</p>
  </footer>

  <script src="script.js"></script>
</body>
</html>
```

- **Style.css**

```
body {
  font-family: Arial, sans-serif;
  margin: 0;
  padding: 0;
  background-color: #f9f9f9;
  color: #333;
}

header {
  background-color: #007acc;
  color: white;
  padding: 20px;
  text-align: center;
}

nav ul {
```

```
  list-style: none;
  padding: 0;
  display: flex;
  justify-content: center;
  margin-top: 10px;
}

nav li {
  margin: 0 15px;
}

nav a {
  color: white;
  text-decoration: none;
  font-weight: bold;
}

main {
  padding: 20px;
  text-align: center;
}

button {
  padding: 10px 20px;
  font-size: 16px;
  cursor: pointer;
```

```
}

footer {
  background-color: #eee;
  text-align: center;
  padding: 15px;
  margin-top: 40px;
}
```

- **Java Script: script.js**

```
function showMessage() {
  const message = document.getElementById("message");
  message.textContent = "Thank you for clicking! Enjoy browsing □";
}
```

Experiment No. 8

❖ **Aim:** Program using Java Script to validate the email address entered by the user (check the presence of "@" & "." character. If this character is missing, the script should display an alert box reporting the error and ask the user to re-enter it again).

Prerequisites: It is required that students should know the basic HTML, Style sheet, and Java Script syntax and methods

❖ **Steps to Create Static Pages**

- **Code**
- **HTML: index.html**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>My Static Website</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <header>
    <h1>Welcome to My Static Website</h1>
    <nav>
      <ul>
        <li><a href="#">Home</a></li>
        <li><a href="#">About</a></li>
        <li><a href="#">Contact</a></li>
```

```
    </ul>
   </nav>
  </header>

  <main>
   <section>
    <h2>Hello, Dear Students!</h2>
    <p>This is a sample static website built using HTML, CSS, and JavaScript.</p>
    <button onclick="showMessage()">Click Me</button>
    <p id="message"></p>

    <h3>Subscribe to our Newsletter</h3>
    <input type="text" id="email" placeholder="Enter your email" />
    <button onclick="validateEmail()">Submit</button>
   </section>
  </main>

  <footer>
   <p>© 2025 FullStackJavaSyllabusSetter</p>
  </footer>

  <script src="script.js"></script>
</body>
</html>
```

- **Style.css**

```
body {
  font-family: Arial, sans-serif;
  margin: 0;
  padding: 0;
  background-color: #f9f9f9;
  color: #333;
}

header {
  background-color: #007acc;
  color: white;
  padding: 20px;
  text-align: center;
}

nav ul {
  list-style: none;
  padding: 0;
  display: flex;
  justify-content: center;
  margin-top: 10px;
}

nav li {
  margin: 0 15px;
```

```
}

nav a {
  color: white;
  text-decoration: none;
  font-weight: bold;
}

main {
  padding: 20px;
  text-align: center;
}

button {
  padding: 10px 20px;
  font-size: 16px;
  cursor: pointer;
}

footer {
  background-color: #eee;
  text-align: center;
  padding: 15px;
  margin-top: 40px;
}
```

- **Java Script: script.js**

```
function showMessage() {
  const message = document.getElementById("message");
  message.textContent = "Thank you for clicking! Enjoy browsing □";
}

function validateEmail() {
  const emailInput = document.getElementById("email");
  const email = emailInput.value;

  if (email.indexOf("@") === -1 || email.indexOf(".") === -1) {
    alert("Invalid email address. Please include '@' and '.' characters.");
    emailInput.focus();
    return false;
  }
  alert("Email is valid!");
  return true;
}
```

Experiment No 9

Aim: **Program based on Document Object Model to change the background color of the web page automatically after every 5 seconds.**

Frontend: HTML + JavaScript (DOM Manipulation)

index.html

```
<!DOCTYPE html>

<html>

<head>

   <meta charset="UTF-8">

   <title>Auto Background Color Change</title>

   <style>

      body {

         transition: background-color 1s ease;

         color: white;

         font-family: Arial, sans-serif;

         text-align: center;

         padding-top: 100px;

      }

   </style>

</head>

<body>

   <h1>Watch the Background Color Change Every 5 Seconds</h1>


   <script>

      const colors = ["#3498db", "#e74c3c", "#2ecc71", "#f39c12", "#9b59b6"];

      let index = 0;
```

```
        setInterval(() => {
            document.body.style.backgroundColor = colors[index];
            index = (index + 1) % colors.length;
        }, 5000);
    </script>
</body>
</html>
```

Backend: Java Servlet (Optional to Serve Page)

Project Structure (Maven)

```
AutoColorChanger/
│
├── src/main/java/com/example/web/ColorServlet.java
├── src/main/webapp/index.html
├── pom.xml
```

ColorServlet.java (optional backend support)

```
package com.example.web;

import java.io.*;
import javax.servlet.*;
```

```
import javax.servlet.http.*;

public class ColorServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        RequestDispatcher dispatcher = request.getRequestDispatcher("index.html");
        dispatcher.include(request, response);
    }
}
```

---

web.xml (Servlet Configuration)

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee" version="3.1">
    <servlet>
        <servlet-name>ColorServlet</servlet-name>
        <servlet-class>com.example.web.ColorServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>ColorServlet</servlet-name>
        <url-pattern>/color</url-pattern>
    </servlet-mapping>
</web-app>
```

pom.xml for Maven

```
<project xmlns="http://maven.apache.org/POM/4.0.0" ...>
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>AutoColorChanger</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>war</packaging>

    <dependencies>
        <dependency>
            <groupId>javax.servlet</groupId>
            <artifactId>javax.servlet-api</artifactId>
            <version>4.0.1</version>
            <scope>provided</scope>
        </dependency>
    </dependencies>
</project>
```

Deployment

1. Use **Apache Tomcat** server.
2. Deploy the WAR file generated by Maven.

Access via: http://localhost:8080/AutoColorChanger/color or directly load index.html.

Experiment No 10

Aim: Program to Build a Stop Watch using Java Script

Implementation code:

The implementation has three files namely

1) HTML file (index.html)
2) CSS file (Style.css)
3) Java Script File (Script.js)

The code for above file is given as follows

HTML File: index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta http-equiv="X-UA-Compatible" content="IE=edge">
   <meta name="viewport" content="width=device-width,
   initial-scale=1.0">
   <title>Stopwatch</title>
   <link rel="stylesheet" href="style.css">
   <link rel="preconnect" href="https://fonts.googleapis.com">
   <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
   <link
href="https://fonts.googleapis.com/css2?family=Nunito:wght@300;500;900&display=swap"
rel="stylesheet">
</head>
<body>
   <div id="timer">00:00:00</div>
   <div id="buttons">
```

```
        <button id="start">Start</button>
        <button id="stop">Stop</button>
        <button id="reset">Reset</button>
    </div>
    <script src="script.js"></script>
</body>
</html>
```

CSS File:style.css

```
body {
background: #360033;  /* fallback for old browsers */
background: -webkit-linear-gradient(to right, #0b8793, #360033);
background: linear-gradient(to right, #0b8793, #360033); /* W3C, IE 10+/ Edge, Firefox 16+,
Chrome 26+, Opera 12+, Safari 7+ */
font-family: 'Nunito', sans-serif;
display: flex;
flex-direction: column;
justify-content: center;
min-height: 100vh;
overflow: hidden;
align-items: center;
}
#timer{
    font-size: 7rem;
    font-weight: 600;
    text-shadow: 2px 2px #5f5f56;
    color: #F1F2B5;
    width: 500px;
    text-align: center;
    margin: 40px auto;
```

```
}

#buttons{
    display: flex;
    justify-content: center;
}
button{
    font-family: 'Nunito', sans-serif;
    background-color: #F1F2B5;
    color: #0b8793;
    border: none;
    font-size: 2rem;
    font-weight: bold;
    padding: 1.5rem 4rem;
    margin: 1rem;
    border-radius: 15px;
    cursor: pointer;
    box-shadow: 2px 2px 10px rgba(58, 51, 51, 0.3);
    transition: all .2s;
}
button:hover{
    background-color: #2c3e50;
    box-shadow: 2px 2px 10px rgba(0, 0, 0, 0.5);
}
button[disabled]{
    opacity: .5;
    cursor: default;
}
@media (max-width: 800px) {
    #timer{
        font-size: 4rem;
```

```
        width: 350px;
    }
    button {
        font-size: calc(20px + (30 - 20) * ((100vw - 320px) / (1200 - 320)));
        padding: calc(20px + (50 - 20) * ((100vw - 320px) / (1200 - 320)));
    }
}
```

Java Script File: script.js

```
const timer = document.getElementById('timer');
const startButton = document.getElementById('start');
const stopButton = document.getElementById('stop');
const resetButton = document.getElementById('reset');

let startTime = 0;
let elapsedTime = 0;
let timerInterval;
function startTimer(){
    startTime = Date.now() - elapsedTime

    timerInterval = setInterval( ()=> {
        elapsedTime = Date.now() - startTime
        timer.textContent = formatTimer(elapsedTime);
    }, 10)

    startButton.disabled = true;
    stopButton.disabled = false;
}
```

```
function stopTimer(){
    clearInterval(timerInterval);
    startButton.disabled = false;
    stopButton.disabled = true;
}

function resetTimer(){
    clearInterval(timerInterval);

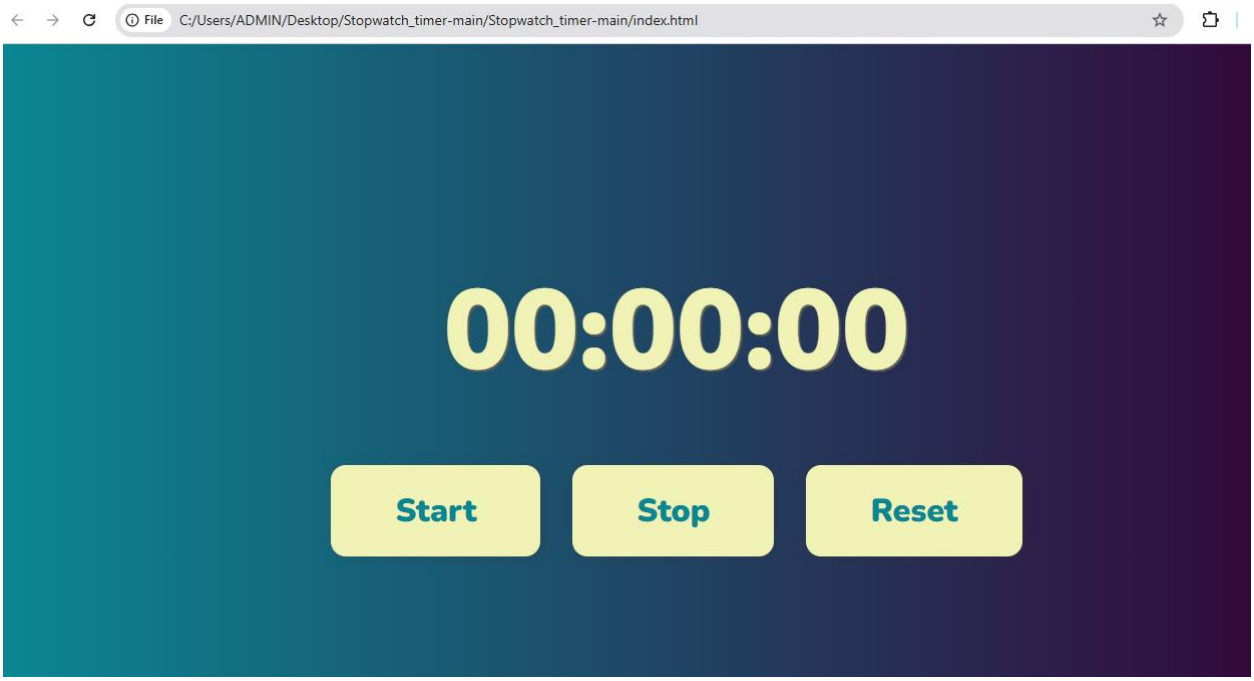
    elapsedTime = 0;
    timer.textContent = "00:00:00";

    startButton.disabled = false;
    stopButton.disabled = false;
}

function formatTimer(elapsedTime){
    const hours = Math.floor(elapsedTime / (1000 * 60 * 60));
    const minutes = Math.floor((elapsedTime % (1000 * 60 * 60)) / (1000 * 60))
    const seconds = Math.floor((elapsedTime % (1000 * 60)) / 1000);
    const mseconds = Math.floor((elapsedTime % 1000) / 10);
    return (
        (hours ? (hours > 9 ? hours : "0" + hours) : "00")
        + ":" +
        (minutes ? (minutes > 9 ? minutes : "0" + minutes) : "00")
        + ":" +
        (seconds ? (seconds > 9 ? seconds : "0" + seconds) : "00")
        + "." +
        (mseconds > 9 ? mseconds : "0" + mseconds));
}
startButton.addEventListener('click', startTimer)
```

```
stopButton.addEventListener('click', stopTimer)
resetButton.addEventListener('click', resetTimer)
```

Output



Experiment No. 10

Aim: Program to Implement Java Script Animation (Text Animation)

Implementation code:

The implementation has three files namely

4) HTML file (index.html)
5) CSS file (Style.css)
6) Java Script File (Script.js)

The code for above file is given as follows

a) HTML file :index.html

```
<!DOCTYPE html>
        <html lang="en">
        <head> <!-- ===( CODE AASHU )=== -->
           <meta charset="UTF-8">
           <meta http-equiv="X-UA-Compatible" content="IE=edge">
           <meta name="viewport" content="width=device-width, initial-scale=1.0">
           <title>CODE AASHU | Very cool animated text rotation</title>
           <link rel="stylesheet" href="style.css">
        </head>
        <body>
           <div id="spiral" class="spiral"></div>
           <div id="spiral2" class="spiral"></div>
           <script src="script.js"></script>
        </body>
</html>
```

b) CSS file: style.css

```
body, html{
            height: 100%;
```

```
    margin: 0;
    display: flex;
    justify-content: center;
    align-items: center;
    background: #040509;
    overflow: hidden;
    font-size: 62.5%
  }

  /* The @property CSS at-rule is part of the CSS Houdini umbrella of APIs. It */
  @property --angle {
     syntax: '<angle>';
     initial-value: 0deg;
     inherits: false;
  }
 .spiral{
    display: flex;
    align-items: center;
    gap: 10px;
    position: absolute;
    color: #e0ecef;
    font-family: "sans-serif";
  }

  @keyframes spiral{
    0%{
      --angle: 0deg;
    }
    100%{
      --angle: 360deg;
    }
  }
 .character{
    font-size: 2.8rem;
    color: white;
    text-transform: uppercase;
    transform: translateY(calc(sin(var(--angle)) * 100px)) scale(calc(cos(var(--
angle)) * 0.5 + 0.5));
    animation: spiral 4s linear infinite;
  }

  @media (max-width: 490px){
    .character{
      font-size: 2.2rem
    }
}
```

c) Java Script File script.js

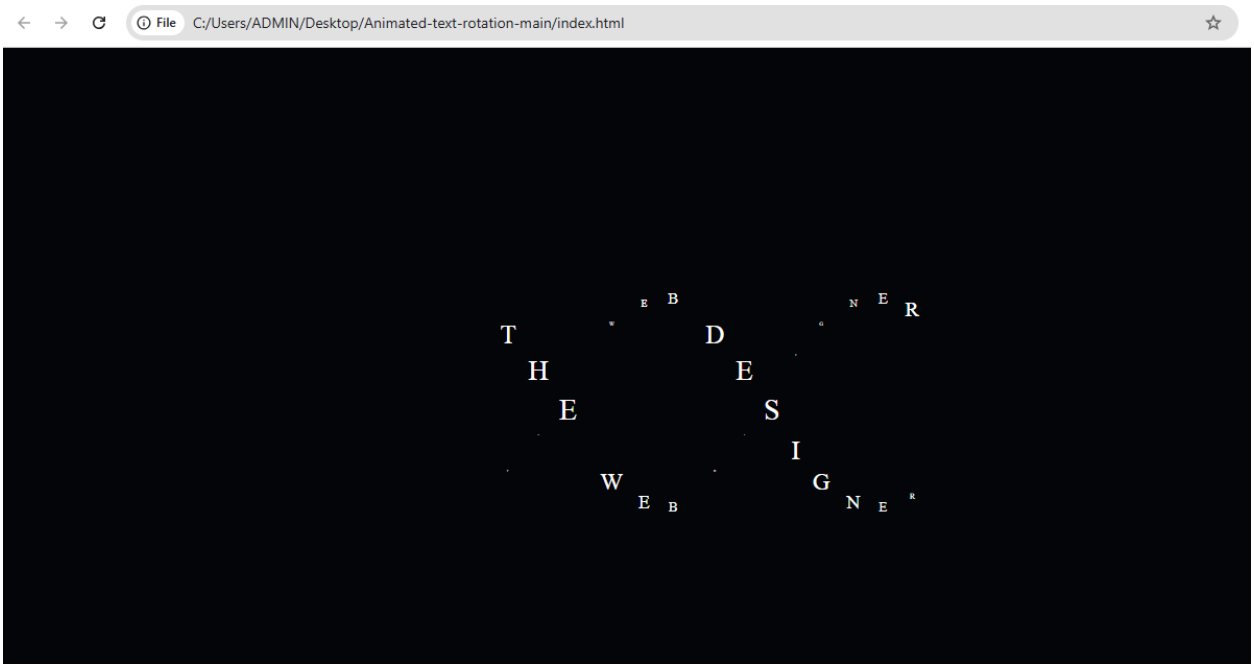
```
let isFirefox = typeof InstallTrigger !== 'undefined';
const words = "The Web Designer";
let ANGLE = 360;
const ANIMATION_DURATION = 4000;
const animation = () => {
  ANGLE -= 1; // Incremento do ângulo
  document.querySelectorAll(".spiral *").forEach((el, i) => {
    const translateY = Math.sin(ANGLE * (Math.PI / 120)) * 100;
    const scale = Math.cos(ANGLE * (Math.PI / 120)) * 0.5 + 0.5;
    const offset = parseInt(el.dataset.offset);
    const delay = i * (ANIMATION_DURATION / 16) - offset;

    setTimeout(() => {
      el.style.transform = `translateY(${translateY}px) scale(${scale})`;
    }, delay);
  });
  requestAnimationFrame(animation);
};
const characters = words.split("").forEach((char, i) => {
  const createElement = (offset) => {
    const div = document.createElement("div");
    div.innerText = char;
    div.classList.add("character");
    div.setAttribute("data-offset", offset);
    div.style.animationDelay = `-${i * (ANIMATION_DURATION / 16) -
offset}ms`
    return div;
  };
  document.querySelector("#spiral").append(createElement(0));
  document
```

```
            .querySelector("#spiral2")
            .append(createElement((isFirefox ? 1 : -1) * (ANIMATION_DURATION /
        2)));
        });

        // @property CSS doesn't work in Firefox, so it must be animated using
        JavaScript.
        if(isFirefox){
          animation();
}
```

Output:



Experiment No 12

Aim: Program for making use of React Hooks that displays four buttons namely, “Red”, “Blue”, “Green”, “Yellow”. On clicking any of these buttons, the code displays the message that you have selected that particular color.

Implementation Steps:

Step 1: Install and Configure React

Prerequisites:

Make sure the following components are installed:

1. **Node.js** (includes npm)
    - Download and install from: https://nodejs.org
    - Verify installation: using Command **node –v or npm –v**

Step 2: Install Create React App (Optional but recommended)

Open Command line and type command

npm install -g create-react-app (Here replace `my-app` with your project name.)

Step 3: Navigate into the Project Folder

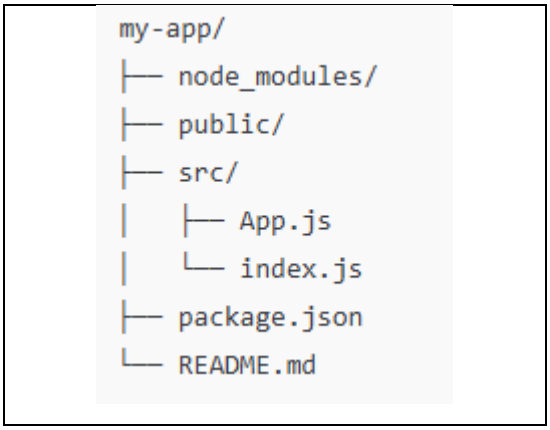
cd my-app

Step 4: Start the Development Server

npm start

This will launch your app in the browser at http://localhost:3000/.

Project Structure Overview



YT Link: https://www.youtube.com/watch?v=1OAt_YtqEfs

Code (App.js)

```
import React, { useState } from 'react';

function App() {

 const [selectedColor, setSelectedColor] = useState('');

 const handleColorClick = (color) => {

  setSelectedColor(color);

 };

 return (

  <div style={{ textAlign: 'center', marginTop: '50px' }}>

   <h2>Select a Color</h2>

   <div style={{ marginBottom: '20px' }}>

    <button onClick={() => handleColorClick('Red')} style={{ margin: '5px',
```

```
backgroundColor: 'red', color: 'white', padding: '10px 20px' }}>Red</button>

      <button onClick={() => handleColorClick('Blue')} style={{ margin: '5px',
backgroundColor: 'blue', color: 'white', padding: '10px 20px' }}>Blue</button>

      <button onClick={() => handleColorClick('Green')} style={{ margin: '5px',
backgroundColor: 'green', color: 'white', padding: '10px 20px' }}>Green</button>

      <button onClick={() => handleColorClick('Yellow')} style={{ margin: '5px',
backgroundColor: 'gold', color: 'black', padding: '10px 20px' }}>Yellow</button>

    </div>

    {selectedColor && (

      <h3>You have selected <span style={{ color: selectedColor.toLowerCase()
}}>{selectedColor}</span>.</h3>

    )}

  </div>

 );}

export default App;
```

Run your app using: npm start

Experiment No. 13

Aim: Creating a Single Page website using the concepts in React like Hooks, Router, Props and States.

Implementation:

App Structure (Pages and Components)

1. `Home` page
2. `About` page
3. `Contact` page

All routed using react-router-dom. A Navbar will let you navigate between them. We'll pass props and use state in Contact page.

📦 Folder Structure

```
src/
├── components/
│   └── Navbar.js
├── pages/
│   ├── Home.js
│   ├── About.js
│   └── Contact.js
├── App.js
└── index.js
```

Step-by-Step Code

Step 1: Install Router

npm install react-router-dom

Step 2: Modify index.js File

index.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import { BrowserRouter } from 'react-router-dom';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <BrowserRouter>
    <App />
  </BrowserRouter>
);
```

Step 3: Modify App.js file

App.js

```
import React from 'react';
import { Routes, Route } from 'react-router-dom';
import Navbar from './components/Navbar';
import Home from './pages/Home';
import About from './pages/About';
import Contact from './pages/Contact';

function App() {
  return (
    <>
      <Navbar />
      <Routes>
        <Route path="/" element={<Home />} />
```

```
      <Route path="/about" element={<About />} />
      <Route path="/contact" element={<Contact name="React User" />} />
    </Routes>
  </>
 );
}
export default App;
```

Step 4: Define the navigations by modifying components/Navbar.js file

```
import React from 'react';
import { Link } from 'react-router-dom';

function Navbar() {
 return (
  <nav style={{ padding: '10px', backgroundColor: '#eee' }}>
    <Link to="/" style={{ marginRight: '10px' }}>Home</Link>
    <Link to="/about" style={{ marginRight: '10px' }}>About</Link>
    <Link to="/contact">Contact</Link>
  </nav>
 );
}
export default Navbar;
```

Step 5: Create an individual Basic Web pages Home.js, About.js and Contact.js

Home.js

```
import React from 'react';
function Home() {
 return (
  <div style={{ padding: '20px' }}>
    <h2>Welcome to the Home Page</h2>
```

```
      <p>This is a simple SPA built with React Router and Hooks.</p>
    </div>
  );
}
export default Home;
```

About.js

```
import React from 'react';

function About() {
  return (
    <div style={{ padding: '20px' }}>
      <h2>About Page</h2>
      <p>This page demonstrates routing in a React app.</p>
    </div>
  );
}
export default About;
```

Contact.js

```
import React, { useState } from 'react';

function Contact({ name }) {
  const [message, setMessage] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();
    alert(`Thanks, ${name}. Message sent: "${message}"`);
    setMessage('');
  };
```

```
  return (
    <div style={{ padding: '20px' }}>
      <h2>Contact Page</h2>
      <form onSubmit={handleSubmit}>
        <textarea
          value={message}
          onChange={(e) => setMessage(e.target.value)}
          placeholder="Type your message..."
          rows="4"
          cols="50"
        />
        <br />
        <button type="submit" style={{ marginTop: '10px' }}>Send Message</button>
      </form>
    </div>
  );
}

export default Contact;
```

Run your app using: npm start

Experiment No. 14

Aim: Program to create a Monolithic Application using SpringBoot

Concept: To understand SpringBoot REST monolithic application using Maven

Database: Example of Book database

REST APIs will cover CRUD operations

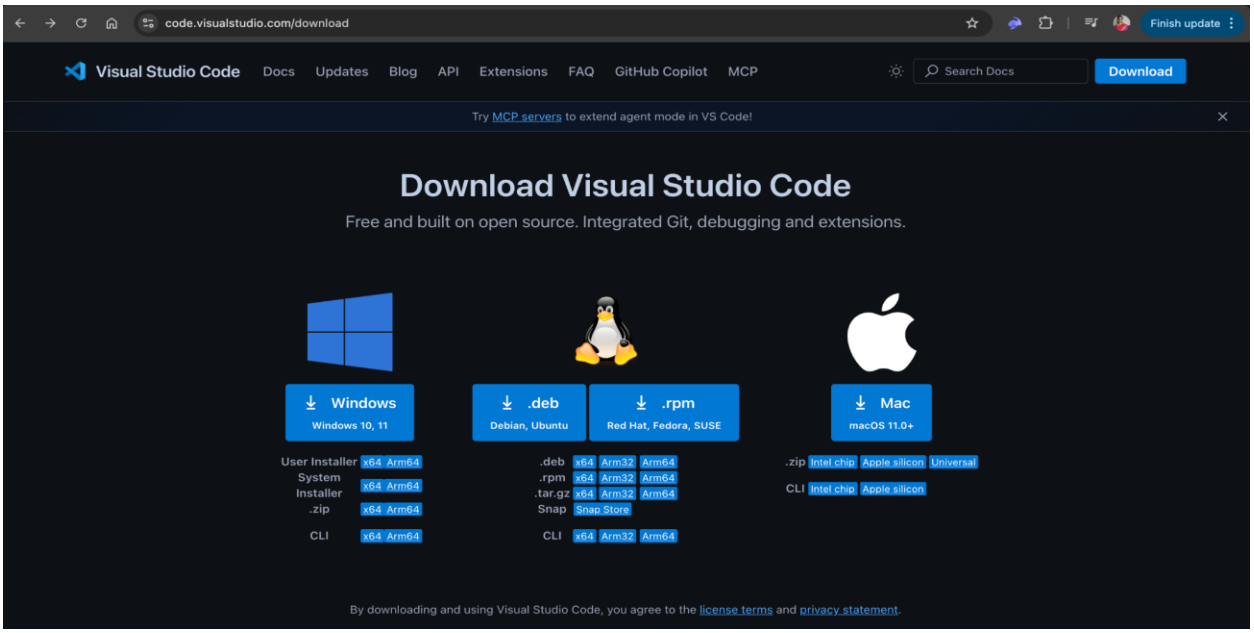
Requirement:

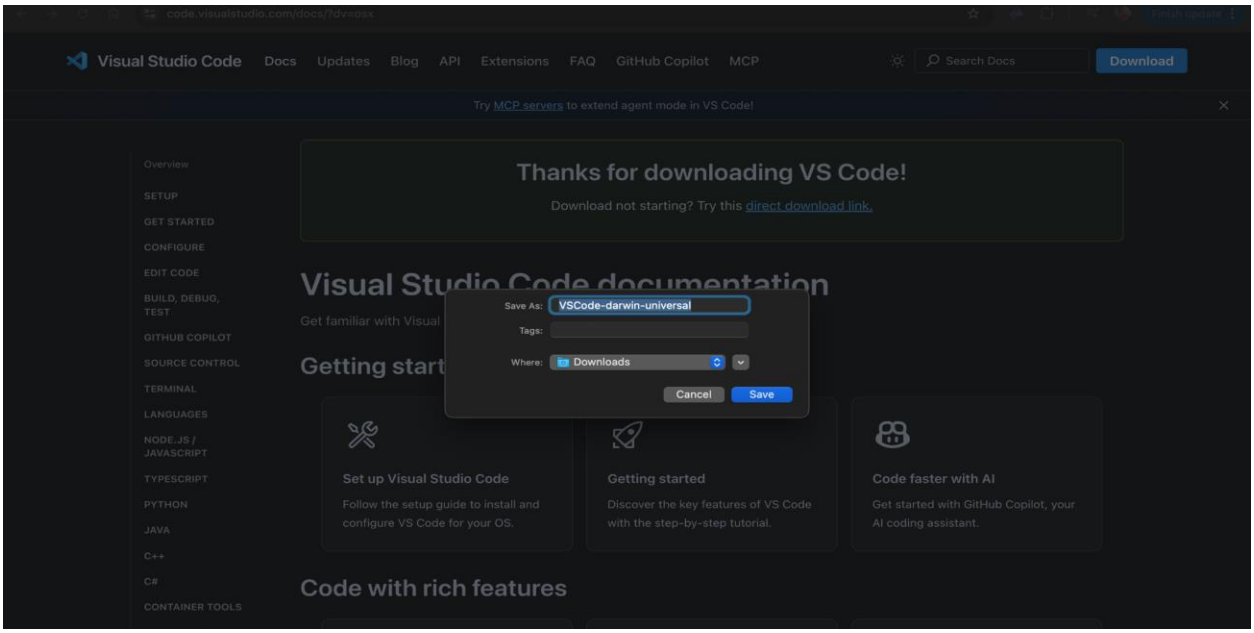
JDK, VS Code, VS Code dependencies, H2 Driver.

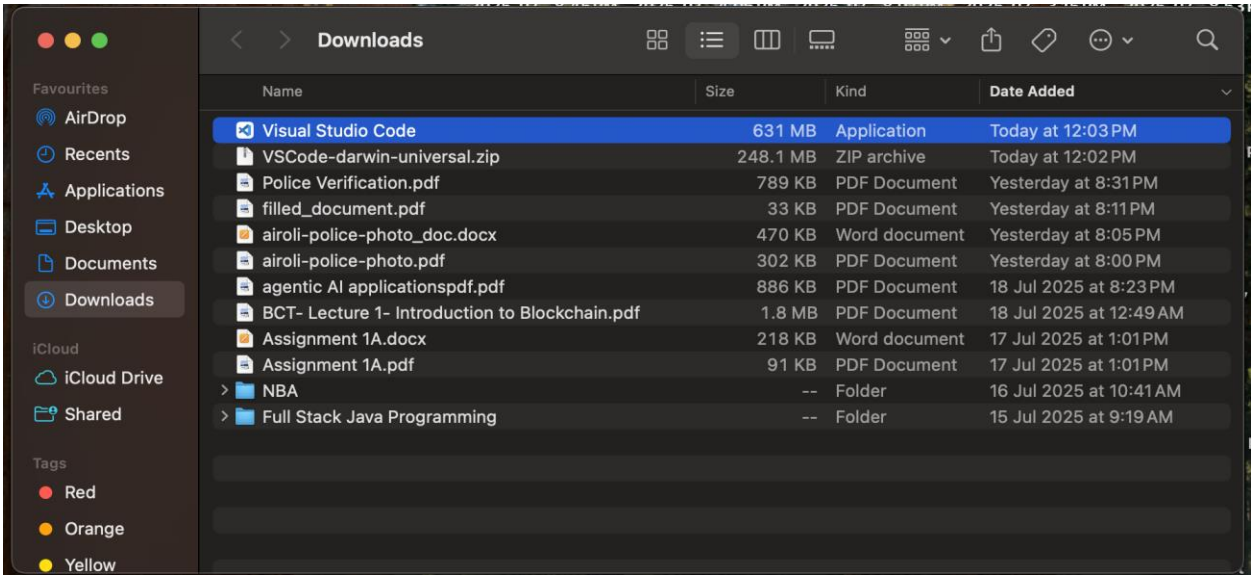
Steps:

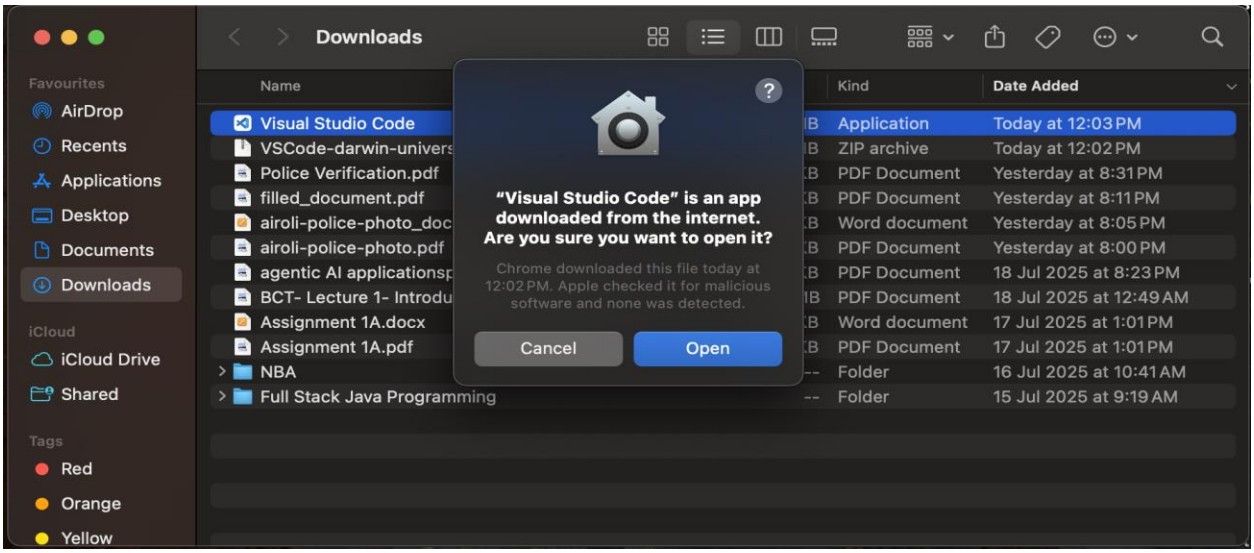
Install VS Code on Windows, Linux, and Mac:

https://code.visualstudio.com/download



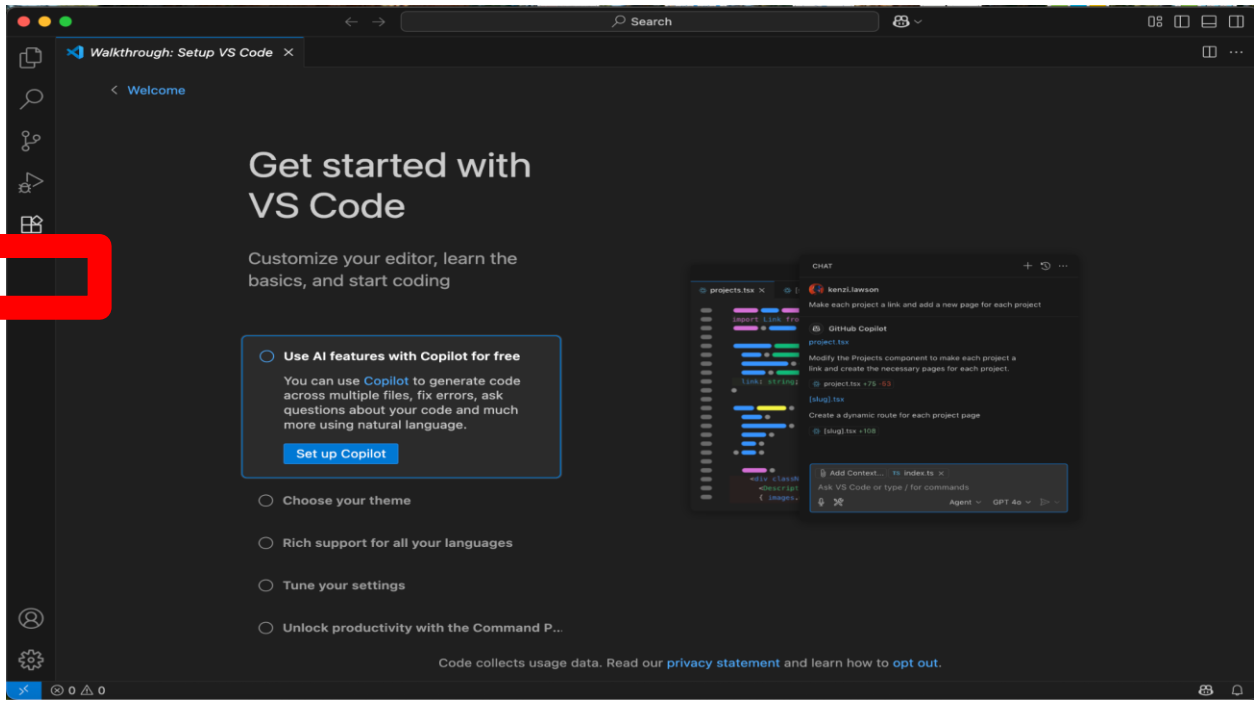

Visual Studio Code
Docs
Updates
Blog
API
Extensions
FAQ
GitHub Copilot
MCP
Search Docs
Download
Try MCP servers to extend agent mode in VS Code!
Thanks for downloading VS Code!
Download not starting? Try this direct download link.
Visual Studio Code documentation
Save As: VSCode-darwin-universal
Tags:
Where: Downloads
Cancel
Save
Set up Visual Studio Code
Follow the setup guide to install and configure VS Code for your OS.
Getting started
Discover the key features of VS Code with the step-by-step tutorial.
Code faster with AI
Get started with GitHub Copilot, your AI coding assistant.
Code with rich features


Downloads
Favourites
AirDrop
Recents
Applications
Desktop
Documents
Downloads
iCloud
iCloud Drive
Shared
Tags
Red
Orange
Yellow
Name
Size
Kind
Date Added
Visual Studio Code 631 MB Application Today at 12:03 PM
VSCode-darwin-universal.zip 248.1 MB ZIP archive Today at 12:02 PM
Police Verification.pdf 789 KB PDF Document Yesterday at 8:31 PM
filled_document.pdf 33 KB PDF Document Yesterday at 8:11 PM
airoli-police-photo_doc.docx 470 KB Word document Yesterday at 8:05 PM
airoli-police-photo.pdf 302 KB PDF Document Yesterday at 8:00 PM
agentic AI applicationspdf.pdf 886 KB PDF Document 18 Jul 2025 at 8:23 PM
BCT- Lecture 1- Introduction to Blockchain.pdf 1.8 MB PDF Document 18 Jul 2025 at 12:49 AM
Assignment 1A.docx 218 KB Word document 17 Jul 2025 at 1:01 PM
Assignment 1A.pdf 91 KB PDF Document 17 Jul 2025 at 1:01 PM
NBA -- Folder 16 Jul 2025 at 10:41 AM
Full Stack Java Programming -- Folder 15 Jul 2025 at 9:19 AM



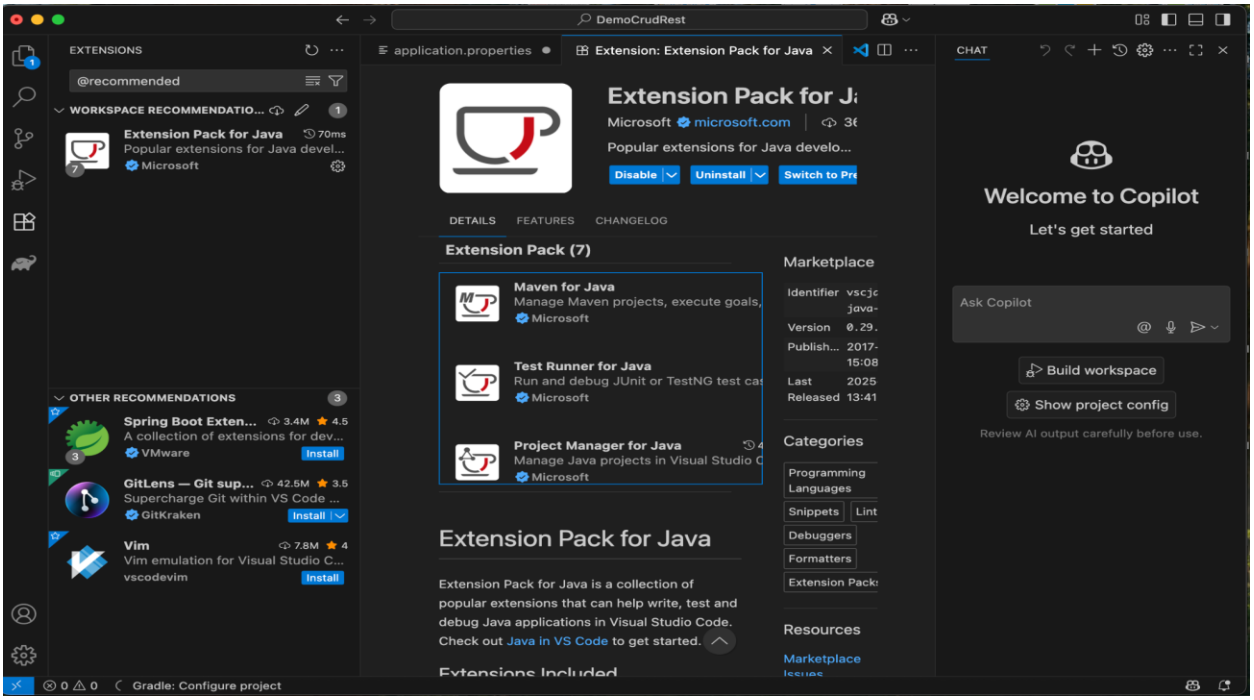
Go to VS Code extensions

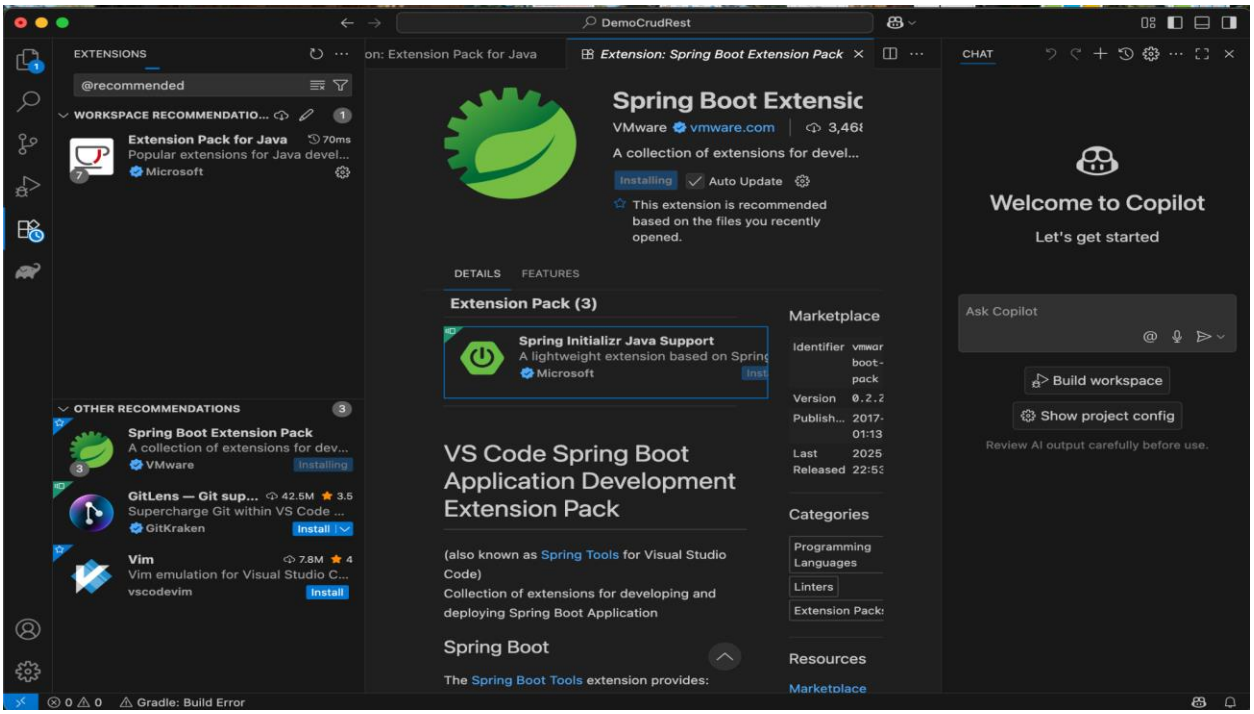
1. Install the extension for Java
2. Install the extension for springboot

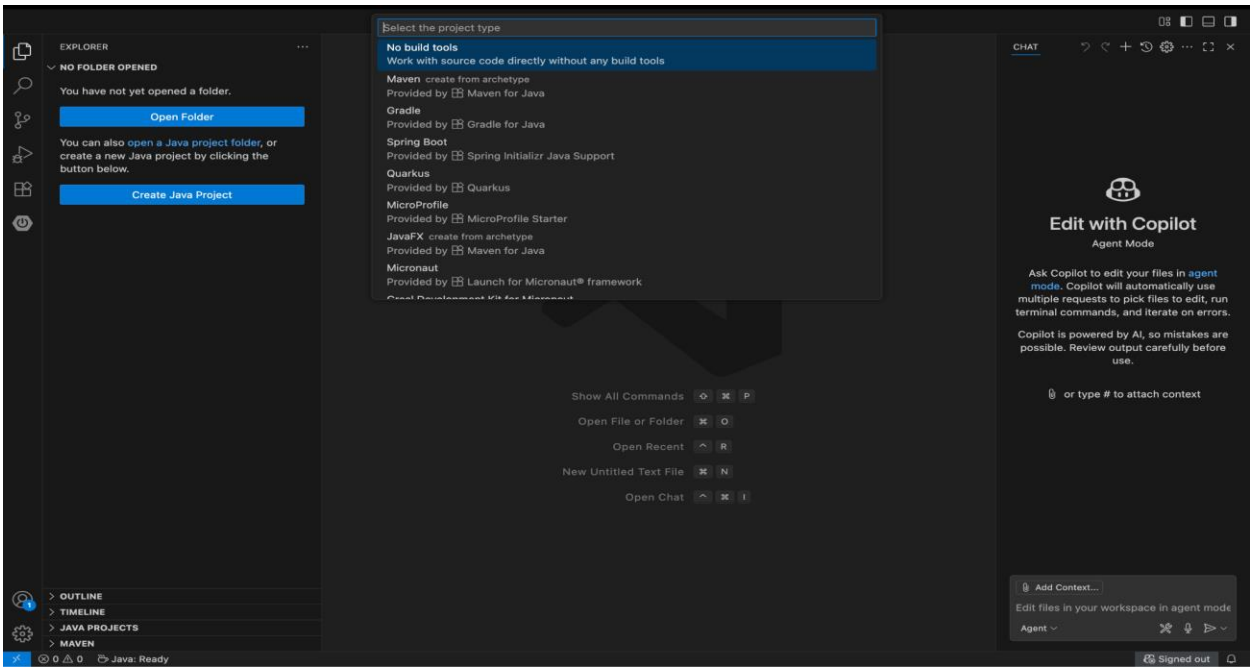


Open the folder and install the recommended extensions, pop right below the window

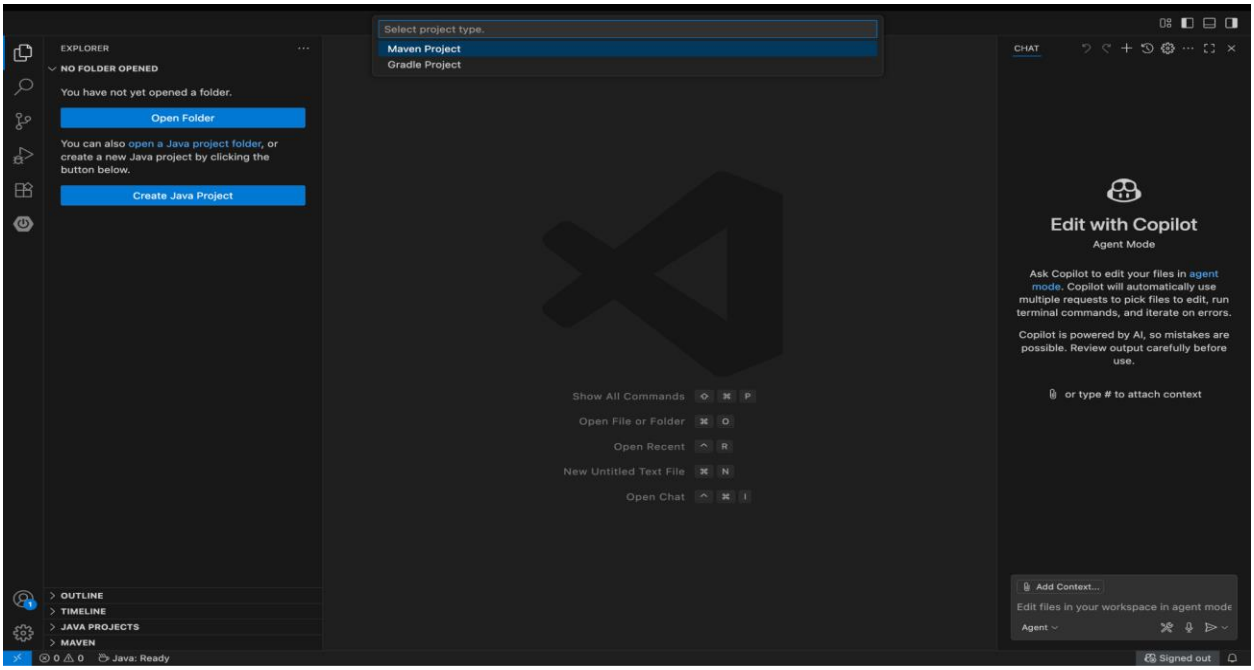
1. Extension pack for Java by Microsoft
2. SpringBoot Extension by VMware







Select SpringBoot



Seectmaven→2.5.3→java→com.example→demo→jar→17→search dependencies

Use dependencies

1. Spring Web: To create a restful application
2. H2: For an in-memory database

3. JPA: Database connectivity
4. Lombok: Java annotation library to avoid writing unnecessary code

Open into folder “anyfoldername”

In the “src→ main→resources→ application.properties” folder, make some property changes

port=8080

Database
1. Url

2. Driver class

3. Username

4. Password

Application.properties when we use the database:

Use port 8080

```
server.port=9090



spring.datasource.url = jdbc:h2:mem:mydb

spring.datasource.driverClassName = org.h2.Driver

spring.datasource.username = sa

spring.datasource.password =

spring.jpa.show-sql = true

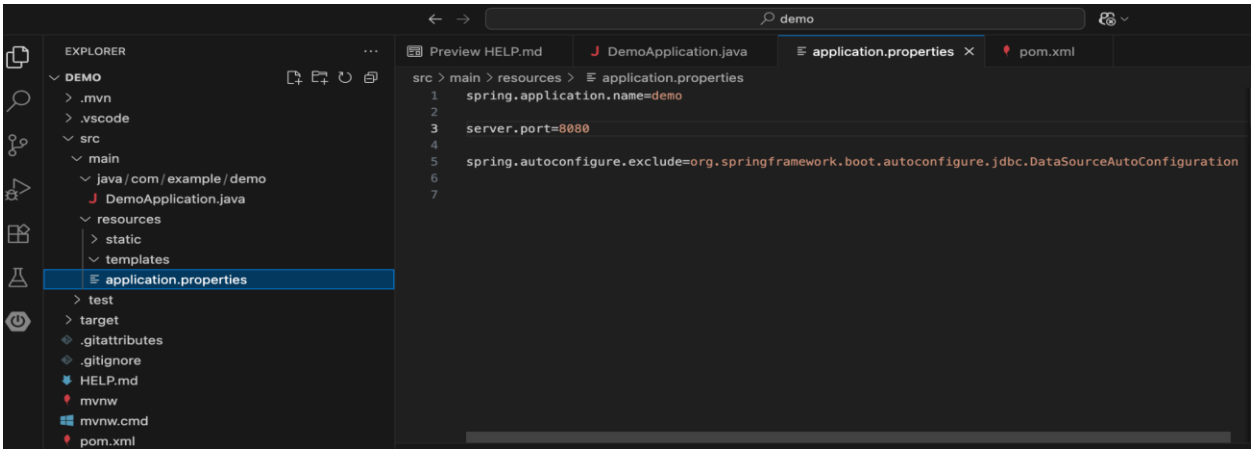
spring.jpa.properties.hibernate.dialect = org.hibernate.dialect.H2Dialect

spring.jpa.hibernate.ddl-auto = update

spring.h2.console.enabled = true

spring.h2.console.path = /h2
```

Application.properties when we do not use the database:



In pom.xml add the dependency to see the local swagger-ui for all rest end points

<dependency>

```
    <groupId>org.springdoc</groupId>

    <!-- for Swaggggggggger ui -->

    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>

    <version>2.3.0</version>

</dependency>
```

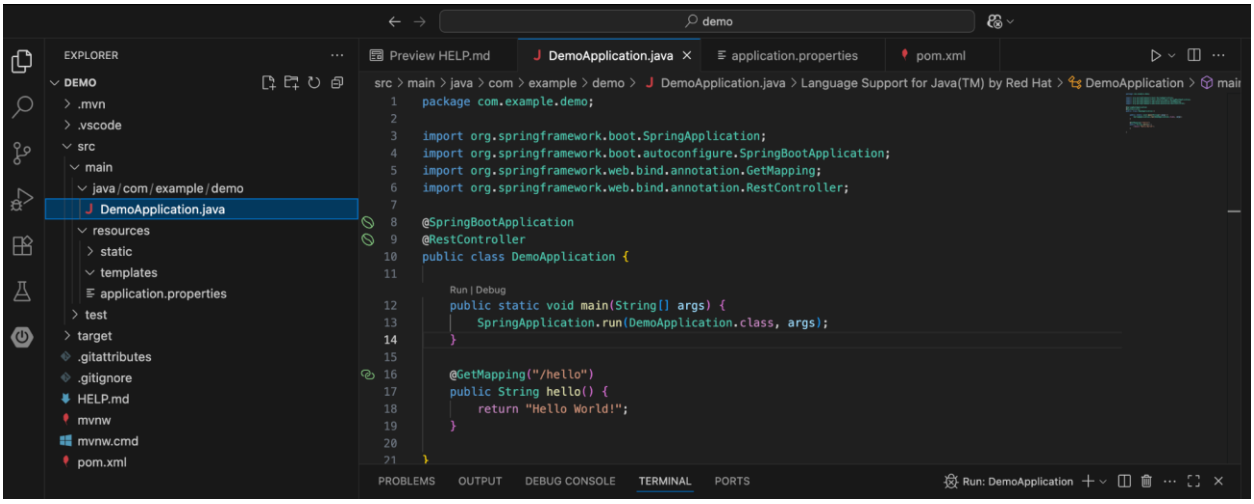
To run the Java file with the main method, **install JDK** on the system and configure it in VS

Code

In src→main→java/com/example/demo

1. DemoApplication.java is already a created file

DemoApplication.java

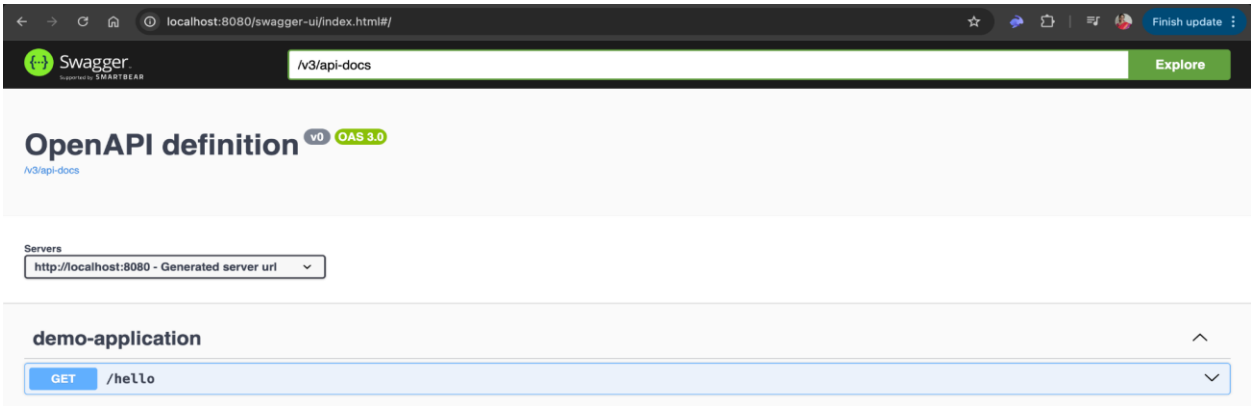


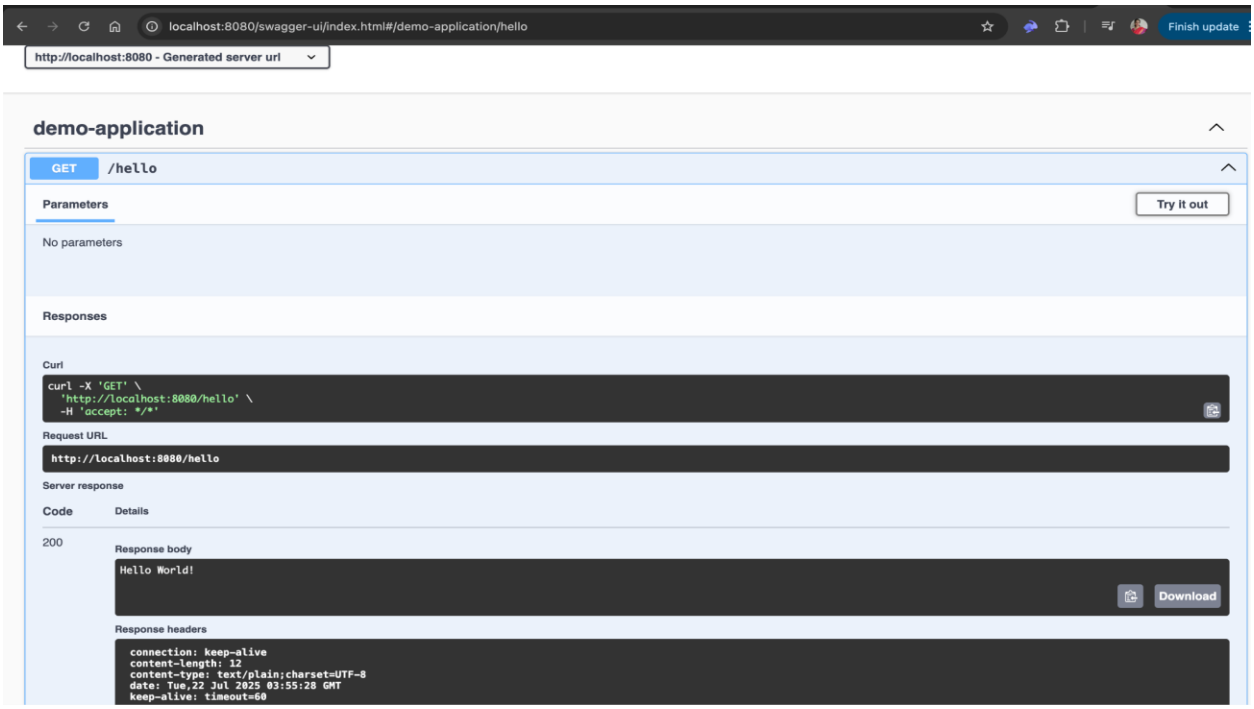
Run the DemoApplication.java file to start the server at localhost:8080/hello

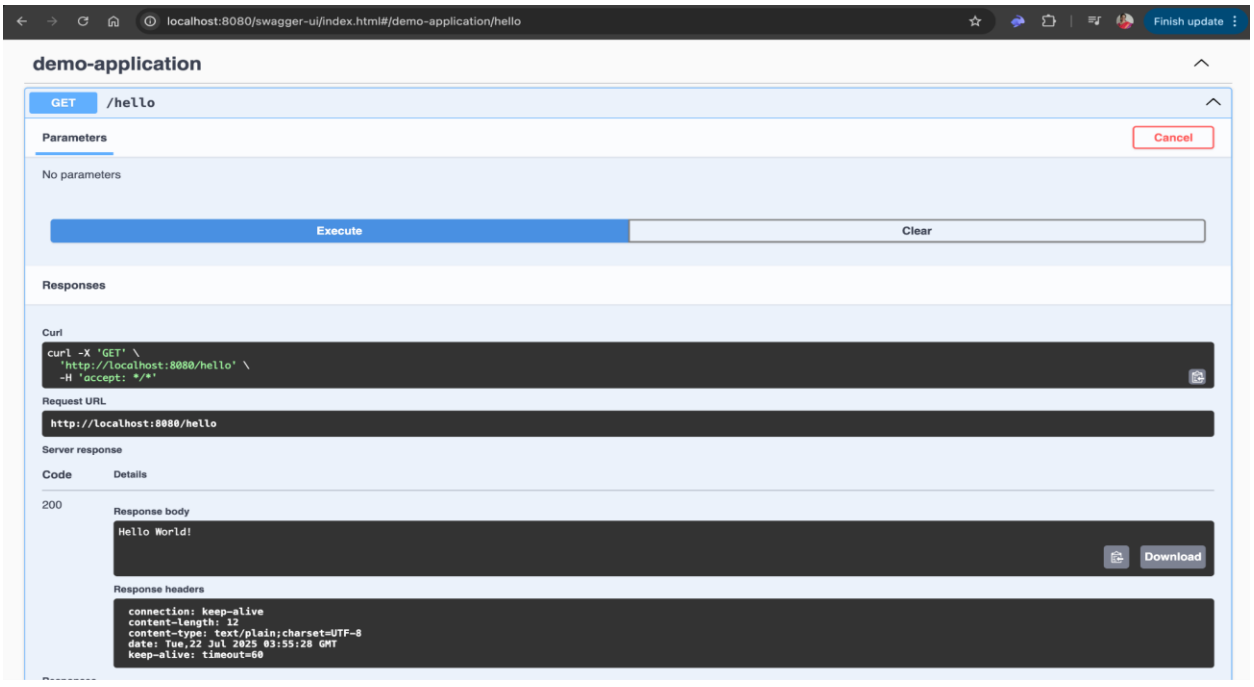
Output:



Open http://localhost:8080/swagger-ui/index.html to see the REST endpoints in the browser




localhost:8080/swagger-ui/index.html#/demo-application/hello
Finish update
http://localhost:8080 - Generated server url
demo-application
GET
/hello
Parameters
Try it out
No parameters
Responses
Curl
curl -X 'GET' \
'http://localhost:8080/hello' \
-H 'accept: */*'
Request URL
http://localhost:8080/hello
Server response
Code
Details
200
Response body
Hello World!
Download
Response headers
connection: keep-alive
content-length: 12
content-type: text/plain;charset=UTF-8
date: Tue,22 Jul 2025 03:55:28 GMT
keep-alive: timeout=60


localhost:8080/swagger-ui/index.html#/demo-application/hello
Finish update
demo-application
GET
/hello
Parameters
Cancel
No parameters
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://localhost:8080/hello' \
-H 'accept: */*'
Request URL
http://localhost:8080/hello
Server response
Code
Details
200
Response body
Hello World!
Download
Response headers
connection: keep-alive
content-length: 12
content-type: text/plain;charset=UTF-8
date: Tue,22 Jul 2025 03:55:28 GMT
keep-alive: timeout=60

Experiment No. 15

Aim: Program for Building RESTful APIs with spring boot

Concept: To understand springBoot REST monolithic application using Maven

Database: Example of Book database/REST APIs will cover CRUD operations

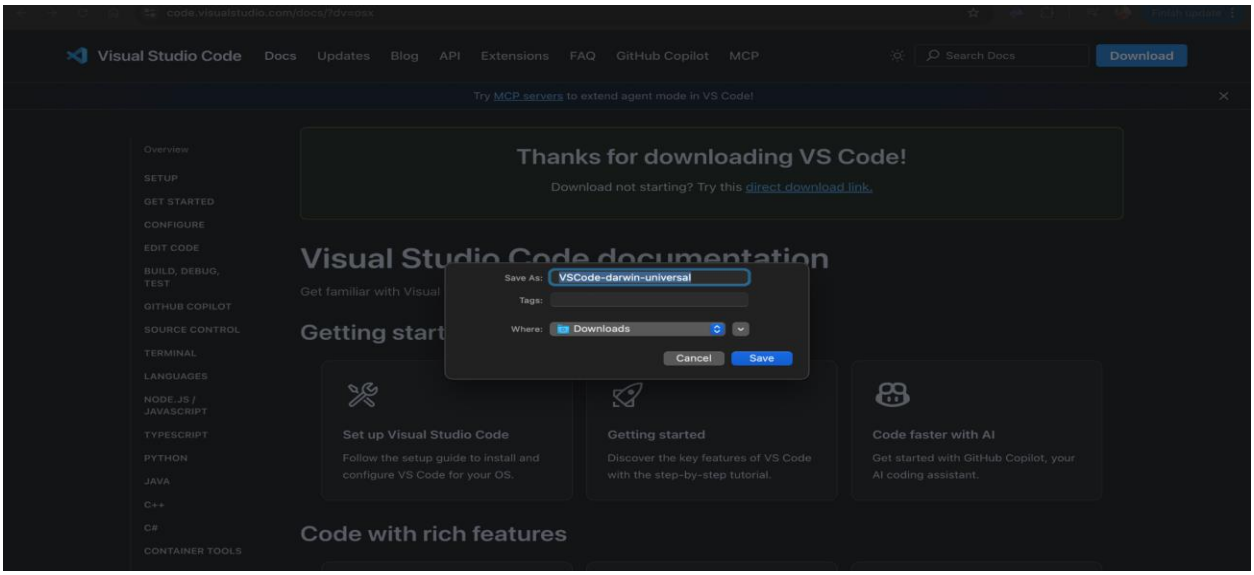
Requirement:

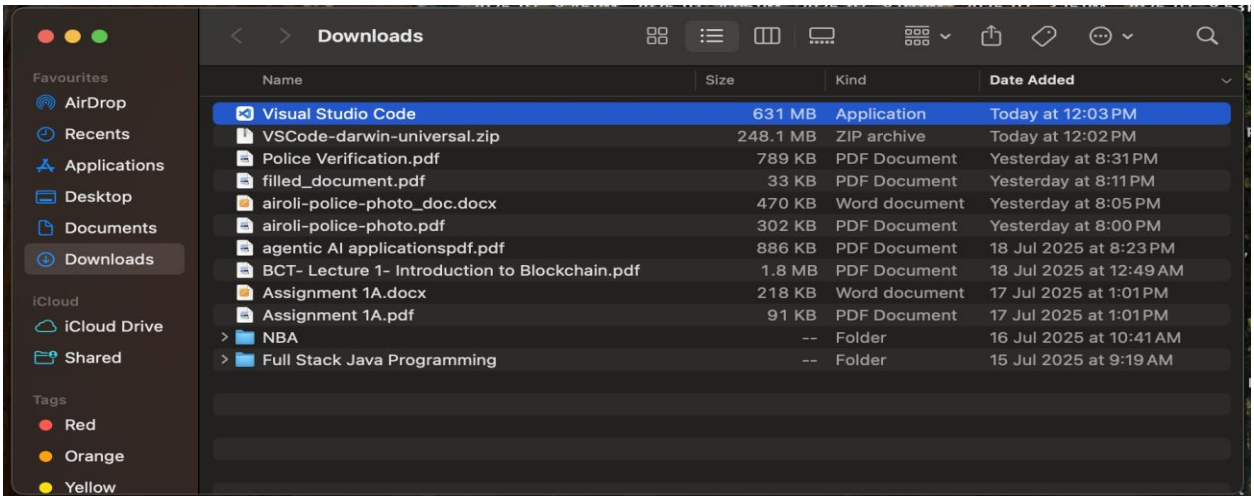
JDK, VS Code, VS Code dependencies, H2 Driver.

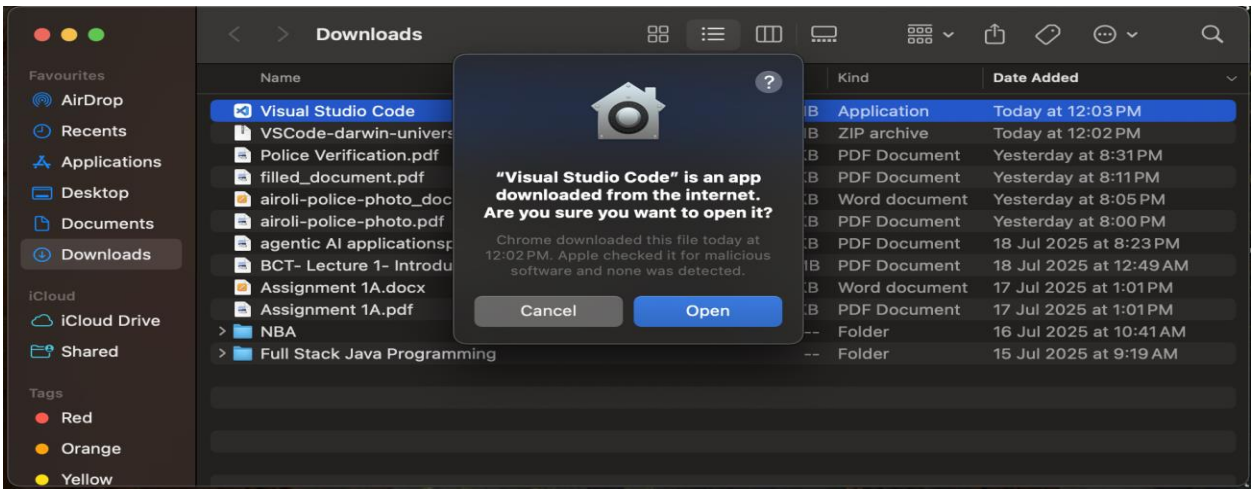
Steps:

Install VS Code on Windows, Linux, and Mac: https://code.visualstudio.com/download



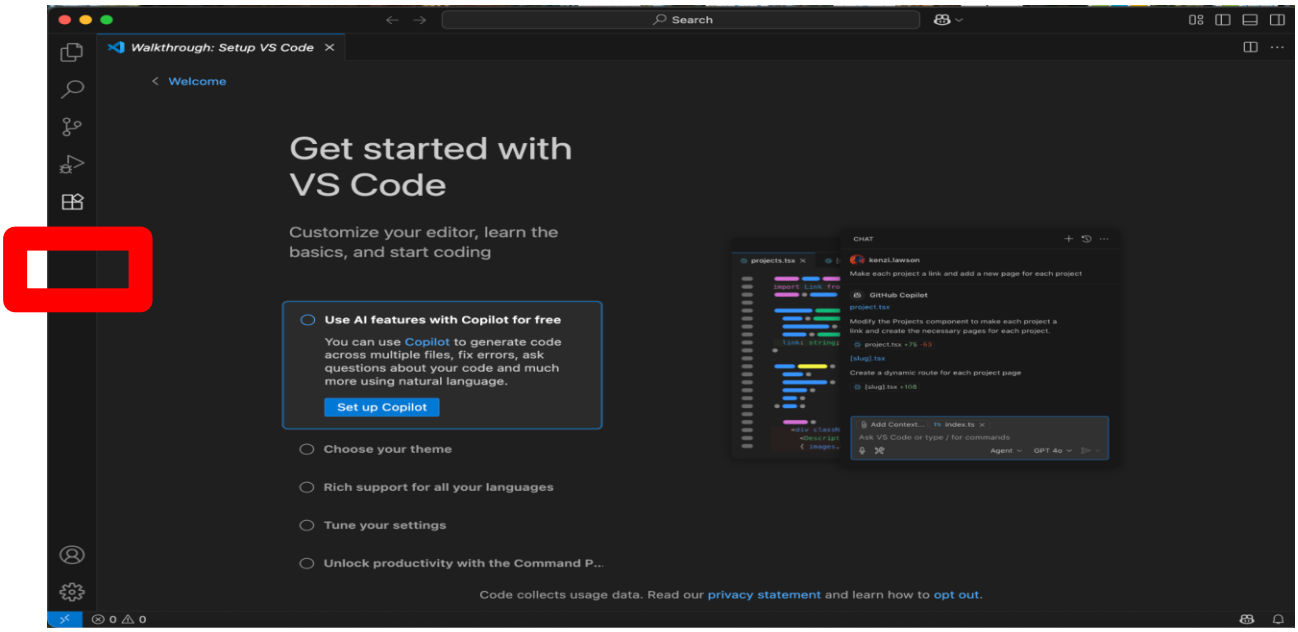

Visual Studio Code
Docs
Updates
Blog
API
Extensions
FAQ
GitHub Copilot
MCP
Search Docs
Download
Try MCP servers to extend agent mode in VS Code!
Thanks for downloading VS Code!
Download not starting? Try this direct download link.
Visual Studio Code documentation
Save As: VSCode-darwin-universal
Tags:
Where: Downloads
Cancel
Save
Set up Visual Studio Code
Follow the setup guide to install and configure VS Code for your OS.
Getting started
Discover the key features of VS Code with the step-by-step tutorial.
Code faster with AI
Get started with GitHub Copilot, your AI coding assistant.
Code with rich features


Downloads
Favourites
AirDrop
Recents
Applications
Desktop
Documents
Downloads
iCloud
iCloud Drive
Shared
Tags
Red
Orange
Yellow
Name
Size
Kind
Date Added
Visual Studio Code 631 MB Application Today at 12:03 PM
VSCode-darwin-universal.zip 248.1 MB ZIP archive Today at 12:02 PM
Police Verification.pdf 789 KB PDF Document Yesterday at 8:31 PM
filled_document.pdf 33 KB PDF Document Yesterday at 8:11 PM
airoli-police-photo_doc.docx 470 KB Word document Yesterday at 8:05 PM
airoli-police-photo.pdf 302 KB PDF Document Yesterday at 8:00 PM
agentic AI applicationspdf.pdf 886 KB PDF Document 18 Jul 2025 at 8:23 PM
BCT- Lecture 1- Introduction to Blockchain.pdf 1.8 MB PDF Document 18 Jul 2025 at 12:49 AM
Assignment 1A.docx 218 KB Word document 17 Jul 2025 at 1:01 PM
Assignment 1A.pdf 91 KB PDF Document 17 Jul 2025 at 1:01 PM
NBA -- Folder 16 Jul 2025 at 10:41 AM
Full Stack Java Programming -- Folder 15 Jul 2025 at 9:19 AM


Downloads
Favourites
AirDrop
Recents
Applications
Desktop
Documents
Downloads
iCloud
iCloud Drive
Shared
Tags
Red
Orange
Yellow
Name
Kind
Date Added
Visual Studio Code Application Today at 12:03 PM
VSCode-darwin-univers ZIP archive Today at 12:02 PM
Police Verification.pdf PDF Document Yesterday at 8:31 PM
filled_document.pdf PDF Document Yesterday at 8:11 PM
airoli-police-photo_doc Word document Yesterday at 8:05 PM
airoli-police-photo.pdf PDF Document Yesterday at 8:00 PM
agentic AI applicationsp PDF Document 18 Jul 2025 at 8:23 PM
BCT- Lecture 1- Introdu PDF Document 18 Jul 2025 at 12:49 AM
Assignment 1A.docx Word document 17 Jul 2025 at 1:01 PM
Assignment 1A.pdf PDF Document 17 Jul 2025 at 1:01 PM
NBA Folder 16 Jul 2025 at 10:41 AM
Full Stack Java Programming Folder 15 Jul 2025 at 9:19 AM
"Visual Studio Code" is an app downloaded from the internet. Are you sure you want to open it?
Chrome downloaded this file today at 12:02 PM. Apple checked it for malicious software and none was detected.
Cancel
Open

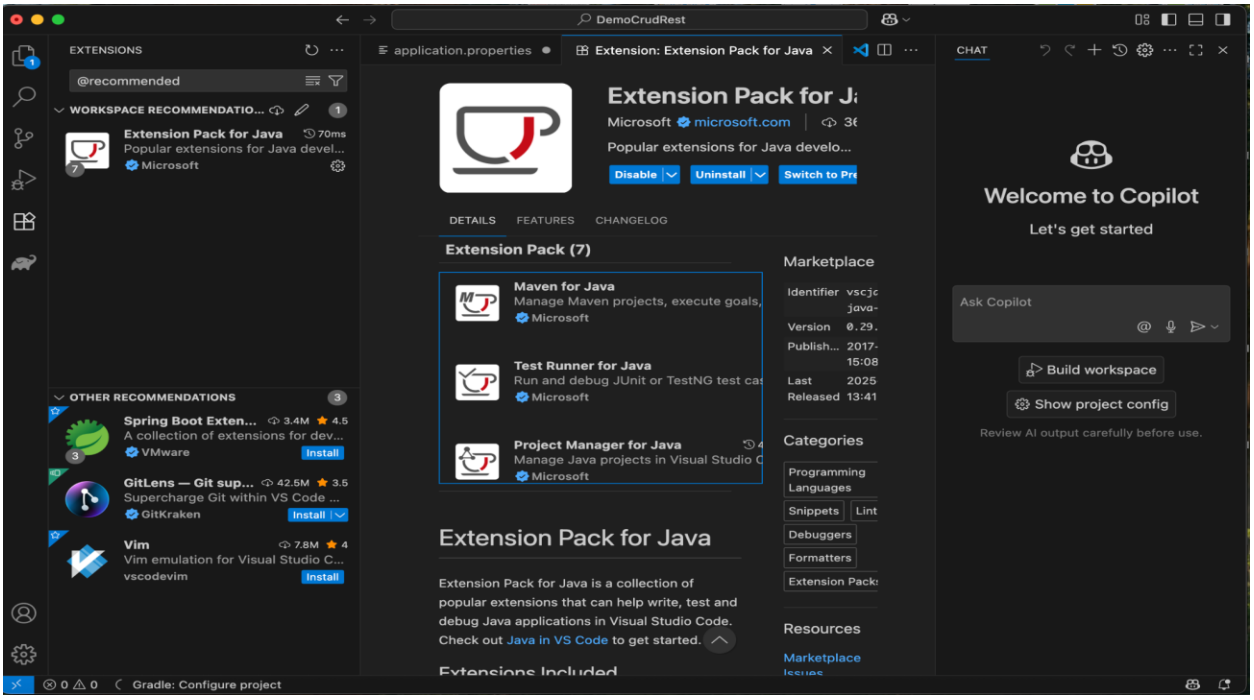
Go to VS Code extensions

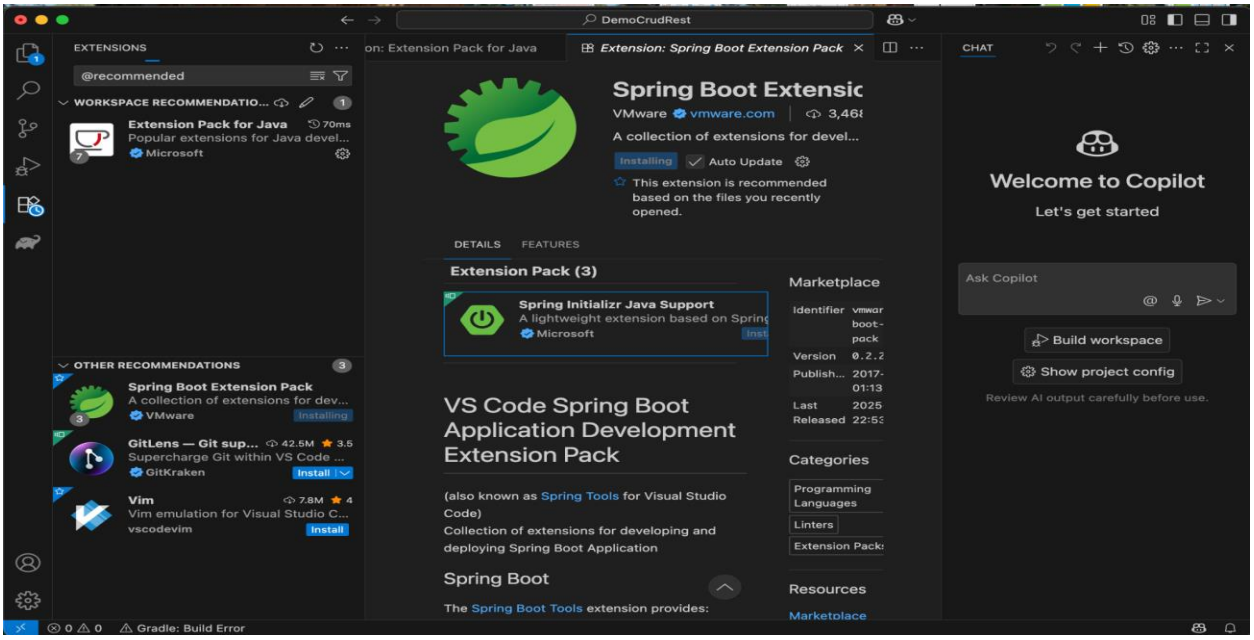
1. Install the extension for Java
2. Install the extension for springboot

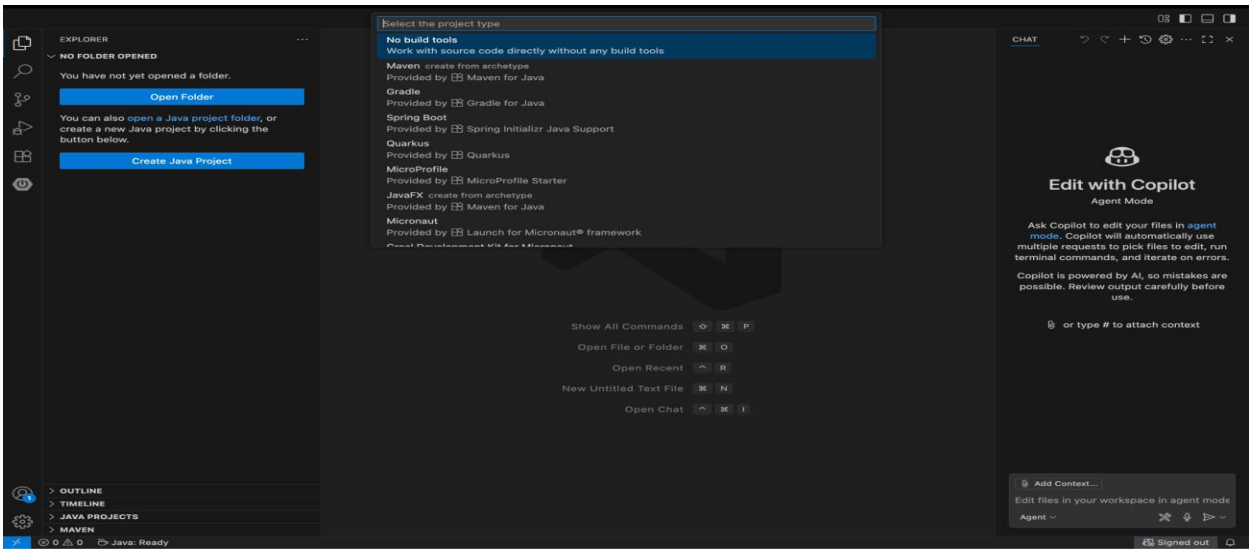


Open the folder and install the recommended extensions, pop right below the window

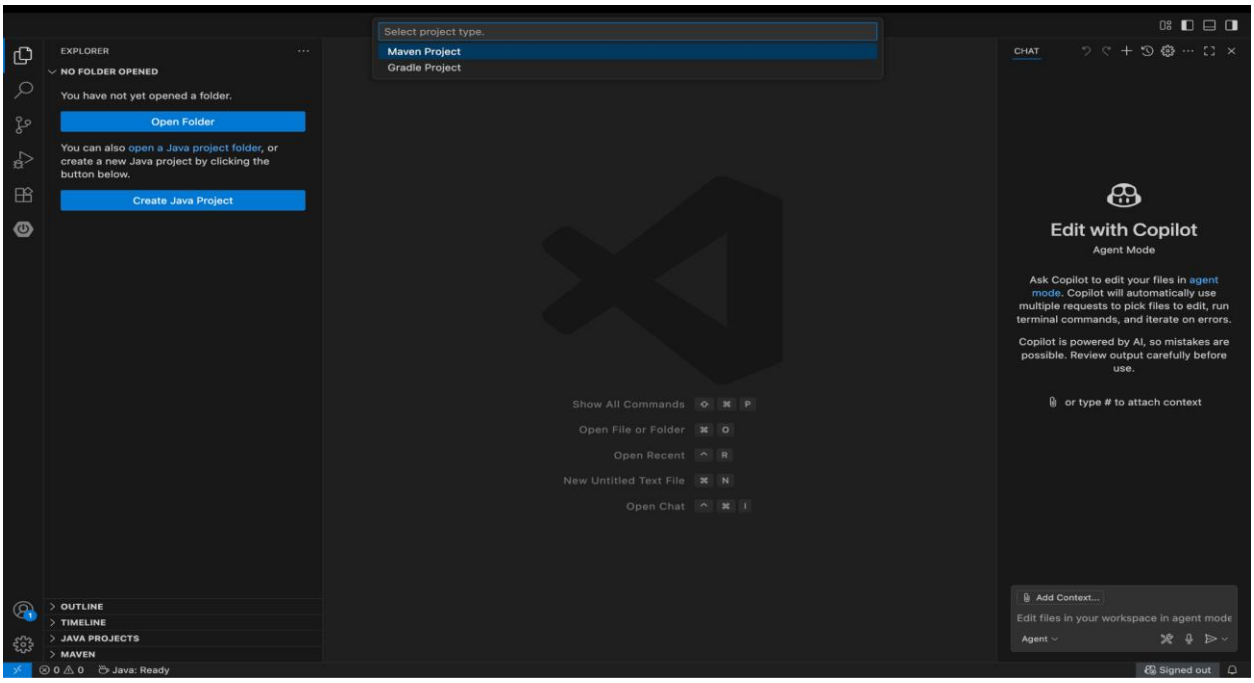
1. Extension pack for Java by Microsoft
2. SpringBoot Extension by VMware


DemoCrudRest
EXTENSIONS
@recommended
WORKSPACE RECOMMENDATIO...
Extension Pack for Java
Popular extensions for Java devel...
Microsoft
application.properties
Extension: Extension Pack for Java
Extension Pack for J
Microsoft microsoft.com
Popular extensions for Java develo...
Disable
Uninstall
Switch to Pre
DETAILS
FEATURES
CHANGELOG
Extension Pack (7)
Maven for Java
Manage Maven projects, execute goals,
Microsoft
Test Runner for Java
Run and debug JUnit or TestNG test cas
Microsoft
Project Manager for Java
Manage Java projects in Visual Studio C
Microsoft
Extension Pack for Java
Extension Pack for Java is a collection of popular extensions that can help write, test and debug Java applications in Visual Studio Code. Check out Java in VS Code to get started.
Extensions Included
Marketplace
Identifier vscjc java-
Version 0.29.
Publish... 2017-15:08
Last Released 2025 13:41
Categories
Programming Languages
Snippets
Lint
Debuggers
Formatters
Extension Packs
Resources
Marketplace
Issues
OTHER RECOMMENDATIONS
Spring Boot Exten... 3.4M 4.5
A collection of extensions for dev...
VMware
Install
GitLens — Git sup... 42.5M 3.5
Supercharge Git within VS Code ...
GitKraken
Install
Vim 7.8M 4
Vim emulation for Visual Studio C...
vscodevim
Install
CHAT
Welcome to Copilot
Let's get started
Ask Copilot
Build workspace
Show project config
Review AI output carefully before use.
Gradle: Configure project


DemoCrudRest
EXTENSIONS
@recommended
WORKSPACE RECOMMENDATIO...
Extension Pack for Java
Popular extensions for Java devel...
Microsoft
on: Extension Pack for Java
Extension: Spring Boot Extension Pack
Spring Boot Extensic
VMware vmware.com 3,468
A collection of extensions for devel...
Installing
Auto Update
This extension is recommended based on the files you recently opened.
DETAILS
FEATURES
Extension Pack (3)
Spring Initializr Java Support
A lightweight extension based on Spring
Microsoft
VS Code Spring Boot Application Development Extension Pack
(also known as Spring Tools for Visual Studio Code)
Collection of extensions for developing and deploying Spring Boot Application
Spring Boot
The Spring Boot Tools extension provides:
Marketplace
Identifier vmwar boot-pack
Version 0.2.2
Publish... 2017-01:13
Last Released 2025 22:53
Categories
Programming Languages
Linters
Extension Packs
Resources
Marketplace
OTHER RECOMMENDATIONS
Spring Boot Extension Pack
A collection of extensions for dev...
VMware
Installing
GitLens — Git sup... 42.5M 3.5
Supercharge Git within VS Code ...
GitKraken
Install
Vim 7.8M 4
Vim emulation for Visual Studio C...
vscodevim
Install
CHAT
Welcome to Copilot
Let's get started
Ask Copilot
Build workspace
Show project config
Review AI output carefully before use.
Gradle: Build Error



Select SpringBoot



Seectmaven→2.5.3→java→com.example→demo→jar→17→search dependencies

Use dependencies

1. Spring Web: To create a restful application
2. H2: For an in-memory database
3. JPA: Database connectivity
4. Lombok: Java annotation library to avoid writing unnecessary code

Open into folder “anyfoldername”

In the “src→ main→resources→ application.properties” folder, make some property changes

port=9090

Database
1. Url

2. Driver class

3. Username

4. Password

application.properties

```
server.port=9090



spring.datasource.url = jdbc:h2:mem:mydb

spring.datasource.driverClassName = org.h2.Driver

spring.datasource.username = sa

spring.datasource.password =

spring.jpa.show-sql = true

spring.jpa.properties.hibernate.dialect = org.hibernate.dialect.H2Dialect

spring.jpa.hibernate.ddl-auto = update
```

```
spring.h2.console.enabled = true

spring.h2.console.path = /h2
```

In pom.xml add the dependency to see the local swagger-ui for all rest end points

<dependency>

```
        <groupId>org.springdoc</groupId>

        <!-- for Swaggggggggger ui -->

        <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>

        <version>2.3.0</version>

    </dependency>
```

To run the Java file with the main method, **install JDK** on the system and configure it in VS Code

In src→main→java/com/example/demo

1. Create a package model and a Java class Book.java in it
2. Create a package repo and a Java interface BookRepo.java in it
3. Create a package controller and a Java class BookController.java in it
4. DemoApplication.java is already a created file

Code:

Book.java

```java
package com.example.demo.model;


import jakarta.persistence.Entity;

import jakarta.persistence.GeneratedValue;

import jakarta.persistence.GenerationType;

import jakarta.persistence.Id;

import jakarta.persistence.Table;

import lombok.AllArgsConstructor;

import lombok.Getter;

import lombok.NoArgsConstructor;

import lombok.Setter;

import lombok.ToString;


@Entity

@Getter

@Setter

@Table(name= "Books")

@NoArgsConstructor

@AllArgsConstructor

@ToString
```

```
public class Book {



  @Id

  @GeneratedValue(strategy = GenerationType.AUTO)

  private long id;

  private String title;

  private String author;




}
```

BookRepo.java

```
package com.example.demo.repo;



import com.example.demo.model.Book;

import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.stereotype.Repository;
```

```
@Repository

public interface BookRepo extends JpaRepository<Book, Long> {

}
```

BookController.java

```
package com.example.demo.controller;



import java.util.ArrayList;

import java.util.Optional;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;

import org.springframework.http.HttpStatus;

import org.springframework.http.ResponseEntity;

import org.springframework.web.bind.annotation.DeleteMapping;

import org.springframework.web.bind.annotation.GetMapping;

import org.springframework.web.bind.annotation.PathVariable;

import org.springframework.web.bind.annotation.PostMapping;

import org.springframework.web.bind.annotation.RequestBody;

import org.springframework.web.bind.annotation.RestController;

import com.example.demo.model.Book;

import com.example.demo.repo.BookRepo;
```

```java
@RestController
public class BookController {

  @Autowired
  private BookRepo bookRepo;

  @GetMapping("/getBooks")
  public ResponseEntity<List<Book>> getAllBooks(){
     try {

        List<Book> bookList = new ArrayList<>();
        bookRepo.findAll().forEach(bookList::add);

        if(bookList.isEmpty()){
           return new ResponseEntity<>(bookList, HttpStatus.NO_CONTENT);
        }

         return new ResponseEntity<>(HttpStatus.OK);

     } catch (Exception e) {
```

```
        return new ResponseEntity<>(HttpStatus.INTERNAL_SERVER_ERROR);
    }


}


@GetMapping("/getBookById/{id}")
public ResponseEntity<Book> getBookById(@PathVariable Long id){


    Optional<Book> bookData =  bookRepo.findById(id);


    if(bookData.isPresent()){
        return new ResponseEntity<>(bookData.get(), HttpStatus.OK);
    }
    return new ResponseEntity<>(HttpStatus.NOT_FOUND);
}


@PostMapping("/addBook")
public ResponseEntity<Book> addBook(@RequestBody Book book) {
    Book bookObj = bookRepo.save(book);
    return new ResponseEntity<>(bookObj, HttpStatus.OK);
}
```

```
  @PostMapping("/updateBookById/{id}")
  public ResponseEntity<Book> updateBookById(@PathVariable Long id, @RequestBody
Book newBookData){
     Optional<Book> oldBookData =  bookRepo.findById(id);
    if(oldBookData.isPresent()){
        Book updatedBookData = oldBookData.get();
        updatedBookData.setTitle(newBookData.getTitle());
        updatedBookData.setAuthor(newBookData.getAuthor());
        Book bookObj = bookRepo.save(updatedBookData);
        return new ResponseEntity<>(bookObj, HttpStatus.OK);
     }
     return new ResponseEntity<>(HttpStatus.NOT_FOUND);
  }
  @DeleteMapping("/deleteBookByID/{id}")
  public ResponseEntity<HttpStatus> deleteBookById(@PathVariable Long id){
    bookRepo.deleteById(id);
     return new ResponseEntity<>(HttpStatus.OK);
  }}
```

DemoApplication.java

```
package com.example.demo;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication

public class DemoApplication {

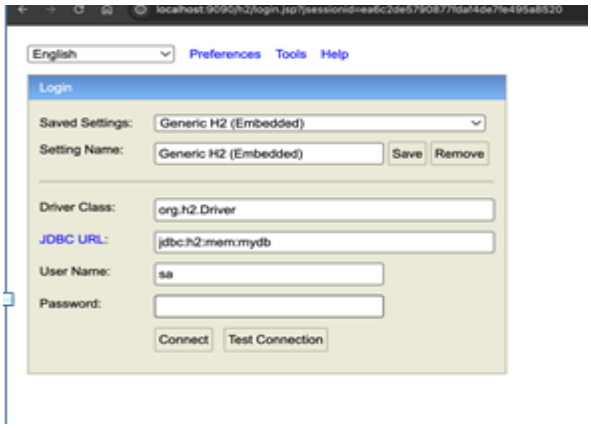
  public static void main(String[] args) {

     SpringApplication.run(DemoApplication.class, args);

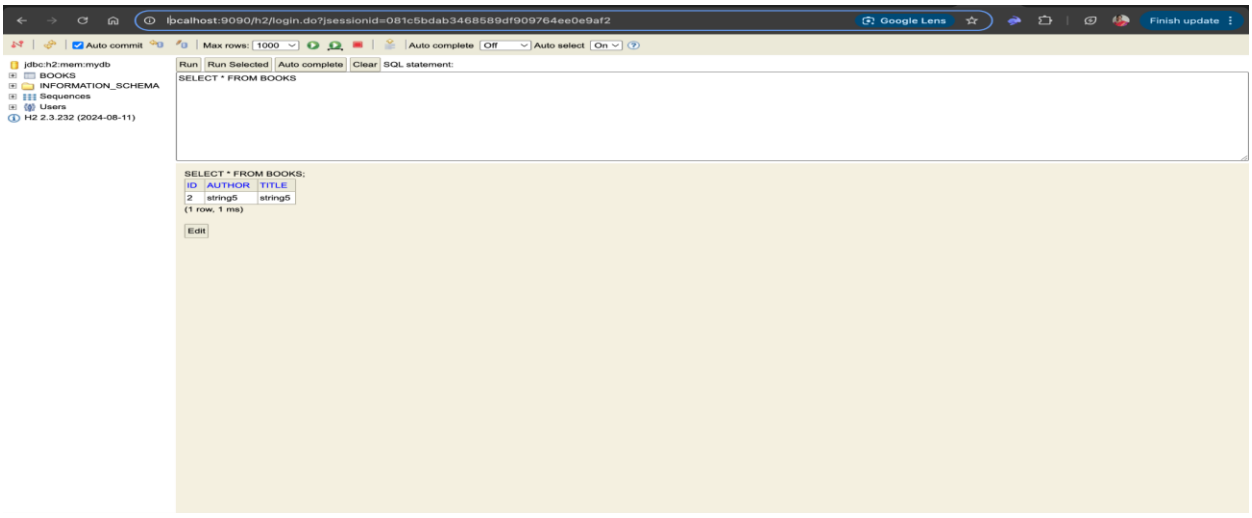
    } }
```

Run the DemoApplication.java file to start the server at localhost:9090

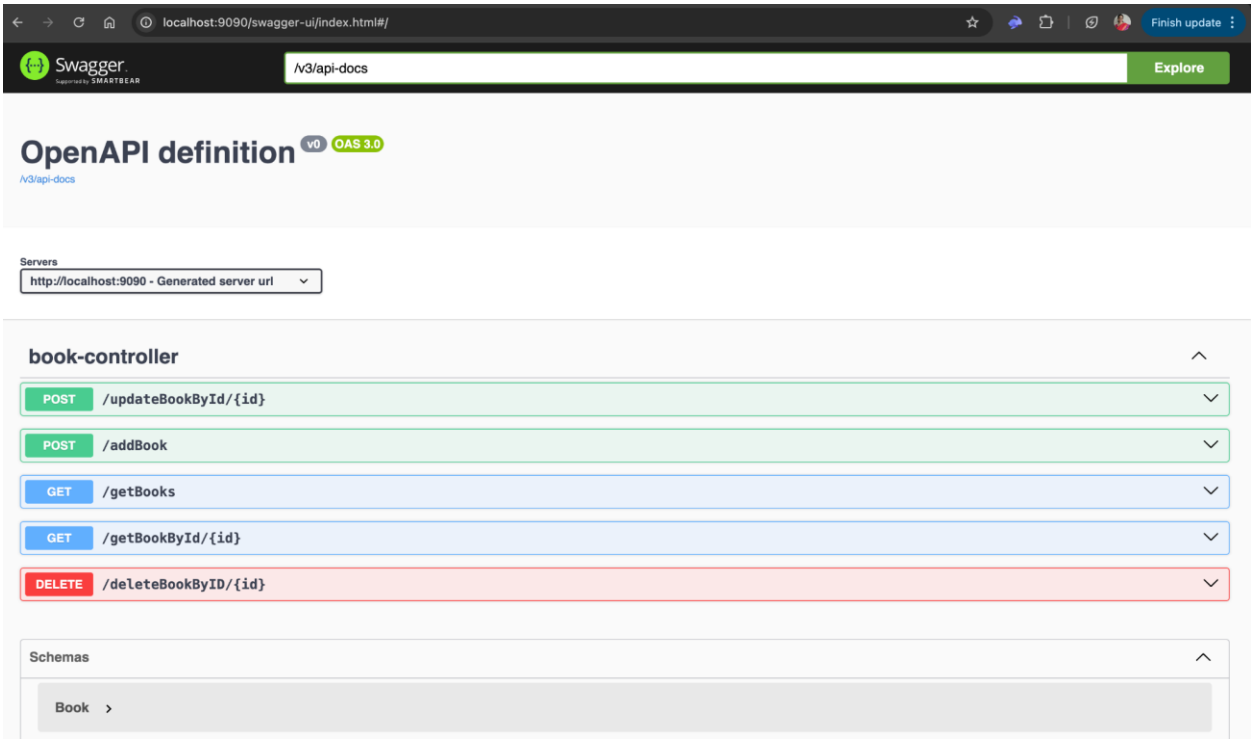
Output:localhost:9090/h2 will give the



Put the JDBC URL as it is there in the application.properties: spring.datasource.url = jdbc:h2:mem:mydb Test the connection and connect with the in-memory database h2



Open http://localhost:9090/swagger-ui/index.html to see the REST endpoints in the browser



Try it each endpoint and execute to see the output, JSON input must be given ID is implicit and auto-incremented, so input must be only the title and author of the book